Project Overview

Our project is centered around predicting "Loss Given Default" (LGD%) of credit card accounts of DCU. LGD% represents the proportion of loss incurred by an institution in the event of a default. We aim to understand and model the factors influencing this crucial metric.

Models Used

Our analysis involves the utilization of two models:

1. **Logistic Regression:**
   - This model is employed for binary classification, specifically to predict whether defaulted customers will repay their obligations to the institution. Key factors impacting recovery chances will be explored using logistic regression.
2. **Linear Regression:**
   - Linear regression is employed to model the relationship between various independent variables and Recovery as well as LGD% . We focus on cases where recovery is greater than 0, aiming to understand the factors influencing the dollar amount recovered.

The subsequent sections will delve into the details of each model, providing insights into the variables considered and the interpretation of the results.

Adding the imports for the Project

In [ ]:
```
import pandas as pd
import decimal
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import subplots
from matplotlib.ticker import StrMethodFormatter
```

Reading the Raw Files

In [ ]:
```
# Read the Excel file
df = pd.read_excel('NewDataset.xlsx')

# Display the DataFrame
df.iloc[:, :,]
```

Out[ ]:

	LoanIDNumber	LOAN_NBR	Charge-Off	Recovery	NCO	FIRST_CO_Month_End_Date	LoanTyp
0	39035	141	12649.67	0.0	12649.67	2015-01-31	6
1	39056	141	8198.21	0.0	8198.21	2013-08-31	6
2	39057	142	5779.70	0.0	5779.70	2014-02-28	6
3	39058	143	1995.31	0.0	1995.31	2021-09-30	6
4	39059	143	1818.41	0.0	1818.41	2019-04-30	6
...	...	...	...	...	...	...	
29796	39731	141	3286.22	0.0	3286.22	2021-08-31	6
29797	10334	142	9928.17	0.0	9928.17	2018-12-31	6
29798	10335	141	9976.79	0.0	9976.79	2017-02-28	6
29799	10336	141	17092.65	0.0	17092.65	2022-10-31	6
29800	39732	142	1500.00	0.0	1500.00	2018-11-30	6

29801 rows × 43 columns

Getting to know the data types

In [ ]:

```
data_types = df.dtypes
print(data_types)
```

```
LoanIDNumber                                     int64
LOAN_NBR                                         int64
Charge-Off                                     float64
Recovery                                       float64
NCO                                            float64
FIRST_CO_Month_End_Date                  datetime64[ns]
LoanType                                         int64
LoanDescription                                 object
MONTH_END_DATE                           datetime64[ns]
BALANCE                                        float64
LIMIT                                          float64
LOAN_TYPE                                        int64
PRODUCT_NAME                                    object
TIMES_DELQ_30_DAYS                               int64
TIMES_DELQ_60_DAYS                               int64
TIMES_DELQ_90_DAYS                               int64
DAYSDEL                                          int64
CREDIT_SCORE                                   float64
CREDIT_SCORE_DATE                        datetime64[ns]
CALC_LOANSTATUS                                 object
MONTHS_OLD                                       int64
SEASONED_OVER_2YR                                int64
BANKPRUPTCY_FLAG                               float64
STATE                                           object
LOYALTY_LEVEL_ID                                 int64
LOYALTY_LEVEL_DESC                              object
MEMBER_TYPE                                     object
BANKRUPTCY_INDEX                               float64
BANKRUPTCY_INDEX_DATE                    datetime64[ns]
HAS_DIRECT_DEP                                   int64
CALC_CURRENT_MONTH_END_SHARE_BALANCE           float64
Calc_DirectIndirect                             object
NEG_SHARE_BALANCE                                int64
LOYALTY_DUMMY_RELATIONSHIP                       int64
LOYALTY_DUMMY_PLUS                               int64
LOYALTY_DUMMY_MEMBER                             int64
ORIG_LOAN_COUNT_LESSTHAN2_DUMMY                  int64
CALC_PRIOR_MONTH_END_SHARE_BALANCE             float64
OPEN_DATE                                datetime64[ns]
CLOSE_DATE                               datetime64[ns]
INT_ACCRUED                                    float64
FINAL_PMT_DATE                                 float64
LGD%                                           float64
dtype: object
```

Changing the data types as per the data dictionary

In [ ]:
```
df['LIMIT'] = df['LIMIT'].astype(float).round(2)
# Replace NaN with a specific value (e.g., 0)
df['LOAN_TYPE'] = df['LOAN_TYPE'].fillna(0).astype(int)
df['PRODUCT_NAME'] = df['PRODUCT_NAME'].astype(str).str.ljust(250)
df['TIMES_DELQ_30_DAYS'] = df['TIMES_DELQ_30_DAYS'].fillna(0).astype(int)
df['TIMES_DELQ_60_DAYS'] = df['TIMES_DELQ_60_DAYS'].fillna(0).astype(int)
df['TIMES_DELQ_90_DAYS'] = df['TIMES_DELQ_90_DAYS'].fillna(0).astype(int)
df['DAYSDEL'] = df['DAYSDEL'].fillna(0).astype(int)
df['CREDIT_SCORE'] = df['CREDIT_SCORE'].fillna(0).astype(int)
df['CALC_LOANSTATUS'] = df['CALC_LOANSTATUS'].astype(str).str.ljust(40)
df['MONTHS_OLD'] = df['MONTHS_OLD'].fillna(0).fillna(0).astype(int)
df['SEASONED_OVER_2YR'] = df['SEASONED_OVER_2YR'].fillna(0).astype(int)
```

```
df['BANKPRUPTCY_FLAG'] = df['BANKPRUPTCY_FLAG'].fillna(0).astype(int)
df['STATE'] = df['STATE'].astype(str).str.ljust(144)
df['LOYALTY_LEVEL_ID'] = df['LOYALTY_LEVEL_ID'].fillna(0).astype(int)
df['LOYALTY_LEVEL_DESC'] = df['LOYALTY_LEVEL_DESC'].astype(str).str.ljust(12)
df['MEMBER_TYPE'] = df['MEMBER_TYPE'].astype(str).str.ljust(8)
df['BANKRUPTCY_INDEX'] = df['BANKRUPTCY_INDEX'].fillna(0).astype(int)
df['HAS_DIRECT_DEP'] = df['HAS_DIRECT_DEP'].fillna(0).astype(int)
df['CALC_CURRENT_MONTH_END_SHARE_BALANCE'] = df['CALC_CURRENT_MONTH_END_SHARE_BALANCE'
df['Calc_DirectIndirect'] = df['Calc_DirectIndirect'].astype(str).str.ljust(8)
df['NEG_SHARE_BALANCE'] = df['NEG_SHARE_BALANCE'].fillna(0).astype(int)
df['LOYALTY_DUMMY_RELATIONSHIP'] = df['LOYALTY_DUMMY_RELATIONSHIP'].fillna(0).astype(i
df['LOYALTY_DUMMY_PLUS'] = df['LOYALTY_DUMMY_PLUS'].fillna(0).astype(int)
df['LOYALTY_DUMMY_MEMBER'] = df['LOYALTY_DUMMY_MEMBER'].fillna(0).astype(int)
df['ORIG_LOAN_COUNT_LESSTHAN2_DUMMY'] = df['ORIG_LOAN_COUNT_LESSTHAN2_DUMMY'].fillna(0
df['INT_ACCRUED'] = df['INT_ACCRUED'].fillna(0).astype(float)
df['CALC_PRIOR_MONTH_END_SHARE_BALANCE'] = df['CALC_PRIOR_MONTH_END_SHARE_BALANCE'].fi
df['MEMBER_TYPE'] = df['MEMBER_TYPE'].astype(str)
```

In [ ]:

```
#Making sure LGD% is between 0 & 1 and replacing anything else with Nan

df.loc[(df['LGD%'] > 1) | (df['LGD%'] < 0), 'LGD%'] = np.nan
```

In [ ]:

```
print(df)
```

```
       LoanIDNumber  LOAN_NBR  Charge-Off  Recovery       NCO  \
0             39035       141    12649.67       0.0  12649.67
1             39056       141     8198.21       0.0   8198.21
2             39057       142     5779.70       0.0   5779.70
3             39058       143     1995.31       0.0   1995.31
4             39059       143     1818.41       0.0   1818.41
...             ...       ...         ...       ...       ...
29796         39731       141     3286.22       0.0   3286.22
29797         10334       142     9928.17       0.0   9928.17
29798         10335       141     9976.79       0.0   9976.79
29799         10336       141    17092.65       0.0  17092.65
29800         39732       142     1500.00       0.0   1500.00

      FIRST_CO_Month_End_Date  LoanType LoanDescription MONTH_END_DATE  \
0                  2015-01-31        66     Credit Card     2014-08-31
1                  2013-08-31        66     Credit Card     2013-03-31
2                  2014-02-28        66     Credit Card     2013-04-30
3                  2021-09-30        66     Credit Card     2021-01-31
4                  2019-04-30        66     Credit Card     2018-11-30
...                       ...       ...             ...            ...
29796              2021-08-31        66     Credit Card     2021-01-31
29797              2018-12-31        66     Credit Card     2018-04-30
29798              2017-02-28        66     Credit Card     2016-07-31
29799              2022-10-31        66     Credit Card     2021-10-31
29800              2018-11-30        66     Credit Card     2017-10-31

        BALANCE  ...  LOYALTY_DUMMY_RELATIONSHIP  LOYALTY_DUMMY_PLUS  \
0      12649.67  ...                           0                   1
1       8198.21  ...                           0                   0
2       5779.70  ...                           1                   0
3       1995.31  ...                           1                   0
4       1818.41  ...                           1                   0
...         ...  ...                         ...                 ...
29796   3286.22  ...                           0                   0
29797   9928.17  ...                           0                   0
29798   9976.79  ...                           1                   0
29799  17325.51  ...                           0                   0
29800   1500.00  ...                           1                   0

      LOYALTY_DUMMY_MEMBER  ORIG_LOAN_COUNT_LESSTHAN2_DUMMY  \
0                        0                                0
1                        1                                0
2                        0                                0
3                        0                                0
4                        0                                0
...                    ...                              ...
29796                    1                                1
29797                    1                                0
29798                    0                                0
29799                    1                                1
29800                    0                                0

       CALC_PRIOR_MONTH_END_SHARE_BALANCE  OPEN_DATE  CLOSE_DATE  INT_ACCRUED  \
0                                  765.46 1998-10-24         NaT        81.94
1                                  -10.80 1997-04-28         NaT        48.81
2                                 1706.60 2000-02-25         NaT        54.19
3                                    9.07 2014-07-09         NaT        21.04
4                                 2952.52 2013-01-22         NaT        19.09
...                                   ...        ...         ...          ...
29796                               38.13 1998-05-28         NaT        39.98
```

```
29797                            137.91 2000-01-12         NaT       925.13
29798                            343.92 2007-12-31         NaT        97.76
29799                            593.24 1993-12-02         NaT       130.17
29800                           -679.06 2014-12-19         NaT       127.79

      FINAL_PMT_DATE     LGD%
0                NaN  1.00000
1                NaN  1.00000
2                NaN  1.00000
3                NaN  1.00000
4                NaN  1.00000
...              ...      ...
29796            NaN  1.00000
29797            NaN  1.00000
29798            NaN  1.00000
29799            NaN  0.98656
29800            NaN  1.00000

[29801 rows x 43 columns]
```

Deleting the Unneccessary columns

In [ ]:
```
import pandas as pd

columns_to_delete = ['LoanDescription', 'LoanType', 'DAYSDEL', 'LOYALTY_LEVEL_DESC', '

df.drop(columns=columns_to_delete, inplace=True)
```

Removing the missing rows

In [ ]:
```
df.dropna(axis=0, inplace=True)
```

In [ ]:
```
print(df)
```

```
       LoanIDNumber  LOAN_NBR  Charge-Off  Recovery       NCO  \
0             39035       141    12649.67       0.0  12649.67
1             39056       141     8198.21       0.0   8198.21
2             39057       142     5779.70       0.0   5779.70
3             39058       143     1995.31       0.0   1995.31
4             39059       143     1818.41       0.0   1818.41
...             ...       ...         ...       ...       ...
29796         39731       141     3286.22       0.0   3286.22
29797         10334       142     9928.17       0.0   9928.17
29798         10335       141     9976.79       0.0   9976.79
29799         10336       141    17092.65       0.0  17092.65
29800         39732       142     1500.00       0.0   1500.00

      FIRST_CO_Month_End_Date MONTH_END_DATE   BALANCE    LIMIT  LOAN_TYPE  \
0                  2015-01-31     2014-08-31  12649.67  14000.0         66
1                  2013-08-31     2013-03-31   8198.21      0.0         66
2                  2014-02-28     2013-04-30   5779.70   7000.0         66
3                  2021-09-30     2021-01-31   1995.31  10000.0         66
4                  2019-04-30     2018-11-30   1818.41      0.0         66
...                       ...            ...       ...      ...        ...
29796              2021-08-31     2021-01-31   3286.22  17000.0         66
29797              2018-12-31     2018-04-30   9928.17      0.0         66
29798              2017-02-28     2016-07-31   9976.79  10000.0         66
29799              2022-10-31     2021-10-31  17325.51  27000.0         66
29800              2018-11-30     2017-10-31   1500.00   1500.0         66

       ... BANKRUPTCY_INDEX  BANKRUPTCY_INDEX_DATE  HAS_DIRECT_DEP  \
0      ...              293             2014-06-30               0
1      ...               82             2013-01-01               1
2      ...              300             2013-03-28               0
3      ...              173             2020-12-31               1
4      ...              301             2018-09-27               1
...    ...              ...                    ...             ...
29796  ...              450             2020-12-31               0
29797  ...              147             2018-01-01               0
29798  ...              231             2016-06-30               1
29799  ...              227             2021-09-30               0
29800  ...              129             2017-06-30               1

       CALC_CURRENT_MONTH_END_SHARE_BALANCE  NEG_SHARE_BALANCE  \
0                                      5.09                  0
1                                     -9.94                  1
2                                   1433.62                  0
3                                    718.84                  0
4                                    290.36                  0
...                                     ...                ...
29796                                 38.32                  0
29797                               1334.57                  0
29798                                 10.28                  0
29799                                285.89                  0
29800                                  0.01                  0

      LOYALTY_DUMMY_MEMBER CALC_PRIOR_MONTH_END_SHARE_BALANCE  OPEN_DATE  \
0                        0                              765.46 1998-10-24
1                        1                              -10.80 1997-04-28
2                        0                             1706.60 2000-02-25
3                        0                                9.07 2014-07-09
4                        0                             2952.52 2013-01-22
...                    ...                                ...        ...
29796                    1                              38.13 1998-05-28
```

```
29797                    1                             137.91 2000-01-12
29798                    0                             343.92 2007-12-31
29799                    1                             593.24 1993-12-02
29800                    0                            -679.06 2014-12-19

       INT_ACCRUED     LGD%
0            81.94  1.00000
1            48.81  1.00000
2            54.19  1.00000
3            21.04  1.00000
4            19.09  1.00000
...            ...      ...
29796        39.98  1.00000
29797       925.13  1.00000
29798        97.76  1.00000
29799       130.17  0.98656
29800       127.79  1.00000

[22168 rows x 33 columns]
```

In [ ]:
```
column_names = df.columns
print(column_names)
```

```
Index(['LoanIDNumber', 'LOAN_NBR', 'Charge-Off', 'Recovery', 'NCO',
       'FIRST_CO_Month_End_Date', 'MONTH_END_DATE', 'BALANCE', 'LIMIT',
       'LOAN_TYPE', 'PRODUCT_NAME', 'TIMES_DELQ_30_DAYS', 'TIMES_DELQ_60_DAYS',
       'TIMES_DELQ_90_DAYS', 'CREDIT_SCORE', 'CREDIT_SCORE_DATE',
       'CALC_LOANSTATUS', 'MONTHS_OLD', 'SEASONED_OVER_2YR',
       'BANKPRUPTCY_FLAG', 'STATE', 'LOYALTY_LEVEL_ID', 'MEMBER_TYPE',
       'BANKRUPTCY_INDEX', 'BANKRUPTCY_INDEX_DATE', 'HAS_DIRECT_DEP',
       'CALC_CURRENT_MONTH_END_SHARE_BALANCE', 'NEG_SHARE_BALANCE',
       'LOYALTY_DUMMY_MEMBER', 'CALC_PRIOR_MONTH_END_SHARE_BALANCE',
       'OPEN_DATE', 'INT_ACCRUED', 'LGD%'],
      dtype='object')
```

For Visualization purpose only:

In [ ]:
```
fp = r"New_trial.xlsx"
df.to_excel(fp, sheet_name='Edit 2')


print("Modified data saved to")
```

```
Modified data saved to
```

Replace the categories with 0 & 1

In [ ]:
```
df['BANKPRUPTCY_FLAG'] = df['BANKPRUPTCY_FLAG'].replace({'FALSE': 0, 'TRUE': 1})
col = df['MEMBER_TYPE'].dtype
print(col)
df['PRODUCT_NAME'] = df['PRODUCT_NAME'].str.strip()
df['MEMBER_TYPE'] = df['MEMBER_TYPE'].replace({'Personal' : 0 , 'Business' : 1})


df['PRODUCT_NAME'] = df['PRODUCT_NAME'].replace({'DCU Visa': 0, 'DCU Rewards Visa': 1}
```

```
object
```

In [ ]:
```
selected_columns = df[['BANKPRUPTCY_FLAG', 'PRODUCT_NAME' , 'MEMBER_TYPE']]
print(selected_columns[['PRODUCT_NAME' , 'MEMBER_TYPE']])
```

```
       PRODUCT_NAME  MEMBER_TYPE
0                 0            0
1                 0            0
2                 0            0
3                 0            0
4                 0            0
...             ...          ...
29796             0            0
29797             0            0
29798             0            0
29799             0            0
29800             0            0

[22168 rows x 2 columns]
```

Created a New column named new_england which consists data only of the New England states

In [ ]:
```
df['STATE'] = df['STATE'].str[0:2]
df.head()
new_england_states = ['MA','NH','ME','CT','VT','RI']
df['new_england'] = df['STATE'].apply(lambda x:0 if x in new_england_states else 1)
```

In [ ]:
```
count_of_0 = (df['new_england'] == 0).sum()
count_of_1 = (df['new_england'] == 1).sum()
print(count_of_0)
print(count_of_1)
```

```
13993
8175
```

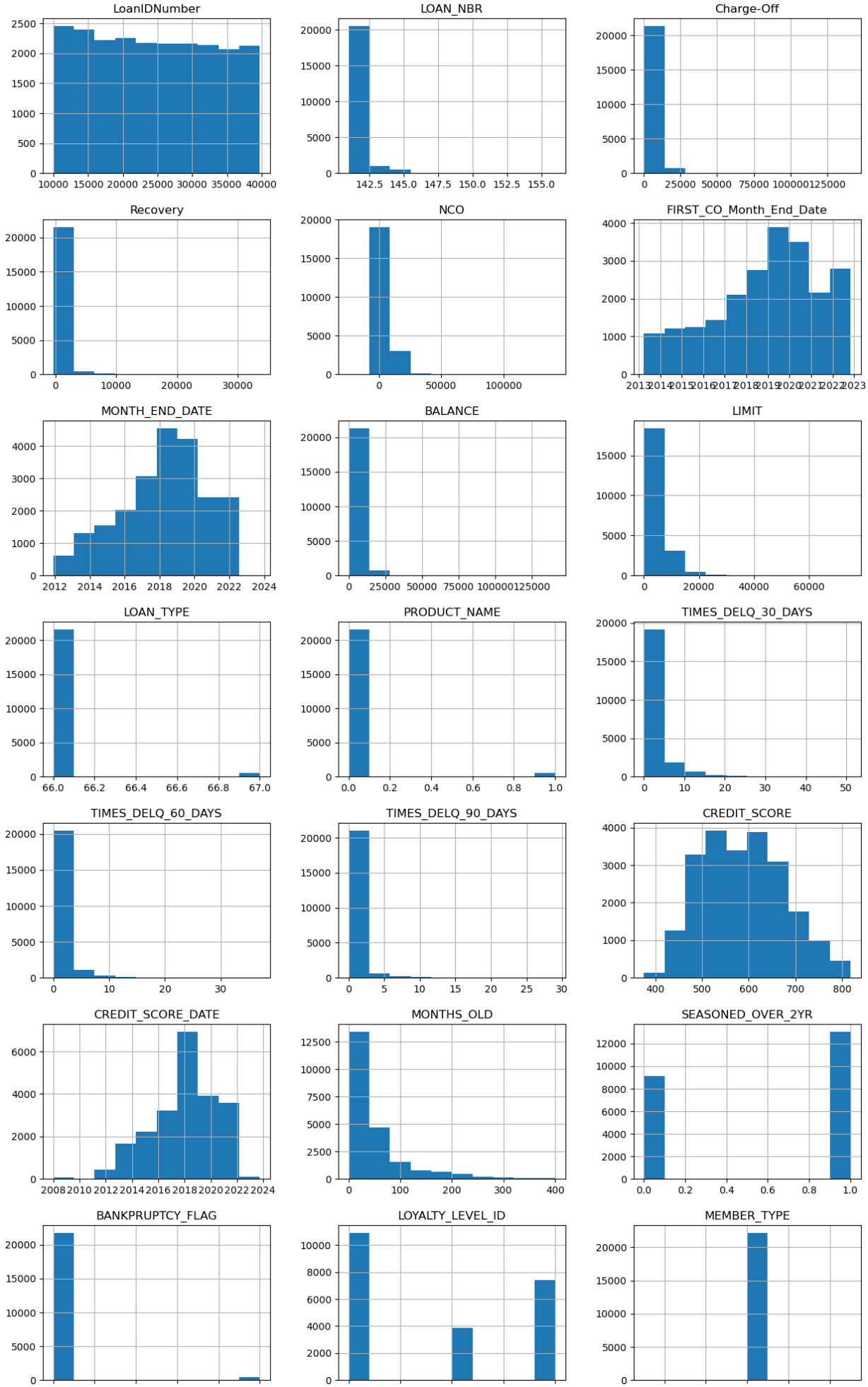
In [ ]:
```
# Dropping State Column as we have already trnsformed it into New-England states.

df.drop(columns=['STATE'], inplace=True)
```

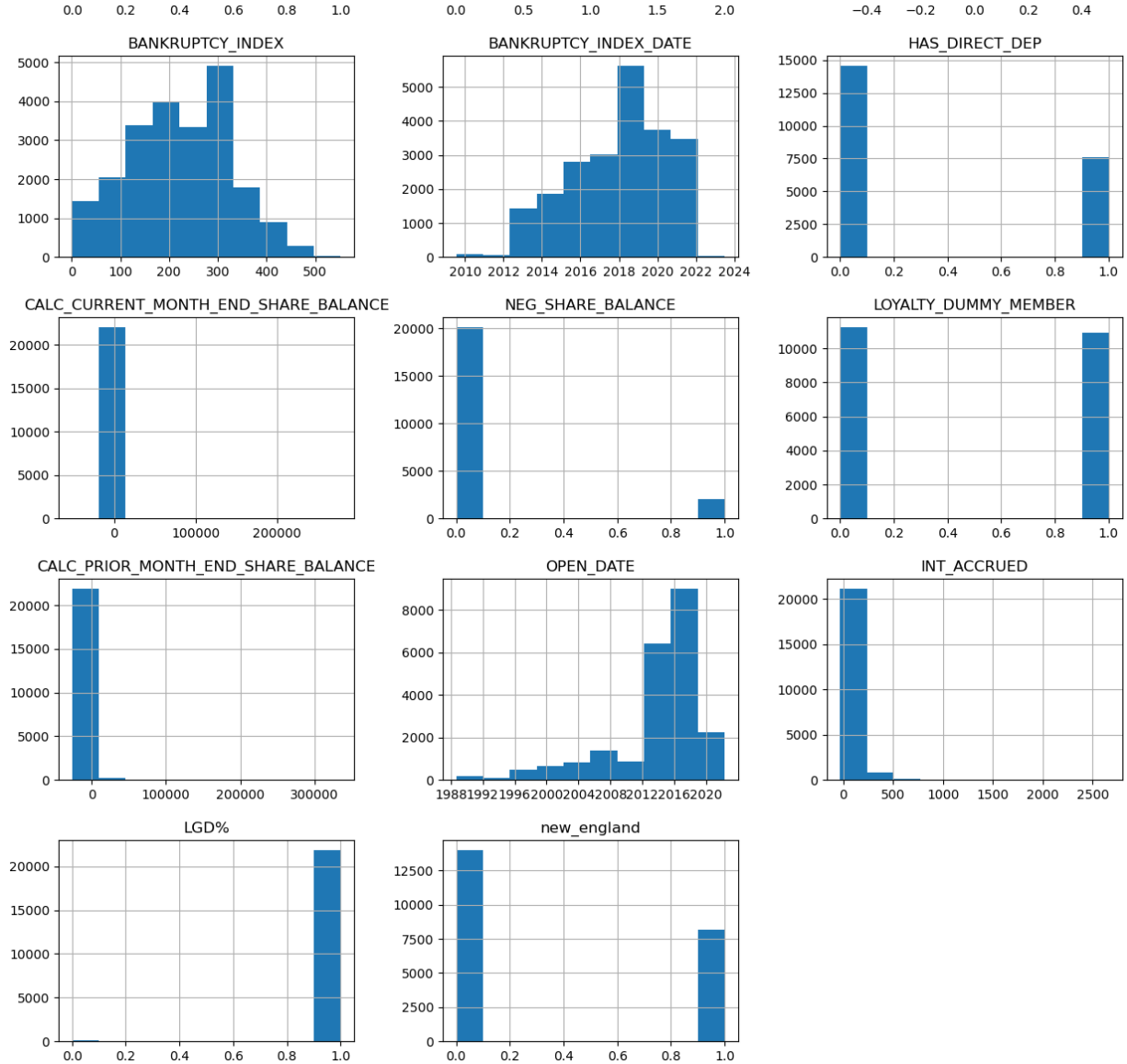
In [ ]:
```
# Write DataFrame to Excel file with sheet name
fp = r"New.xlsx"
df.to_excel(fp, sheet_name='Edit 1')


print("Modified data saved to")
```

```
Modified data saved to
```

In [ ]:
```
df.hist(figsize=(15, 40), layout=(11,3))
```

Out[ ]:
```
array([[<AxesSubplot:title={'center':'LoanIDNumber'}>,
        <AxesSubplot:title={'center':'LOAN_NBR'}>,
        <AxesSubplot:title={'center':'Charge-Off'}>],
       [<AxesSubplot:title={'center':'Recovery'}>,
        <AxesSubplot:title={'center':'NCO'}>,
        <AxesSubplot:title={'center':'FIRST_CO_Month_End_Date'}>],
       [<AxesSubplot:title={'center':'MONTH_END_DATE'}>,
        <AxesSubplot:title={'center':'BALANCE'}>,
        <AxesSubplot:title={'center':'LIMIT'}>],
       [<AxesSubplot:title={'center':'LOAN_TYPE'}>,
        <AxesSubplot:title={'center':'PRODUCT_NAME'}>,
        <AxesSubplot:title={'center':'TIMES_DELQ_30_DAYS'}>],
       [<AxesSubplot:title={'center':'TIMES_DELQ_60_DAYS'}>,
        <AxesSubplot:title={'center':'TIMES_DELQ_90_DAYS'}>,
        <AxesSubplot:title={'center':'CREDIT_SCORE'}>],
       [<AxesSubplot:title={'center':'CREDIT_SCORE_DATE'}>,
        <AxesSubplot:title={'center':'MONTHS_OLD'}>,
        <AxesSubplot:title={'center':'SEASONED_OVER_2YR'}>],
       [<AxesSubplot:title={'center':'BANKPRUPTCY_FLAG'}>,
        <AxesSubplot:title={'center':'LOYALTY_LEVEL_ID'}>,
        <AxesSubplot:title={'center':'MEMBER_TYPE'}>],
       [<AxesSubplot:title={'center':'BANKRUPTCY_INDEX'}>,
        <AxesSubplot:title={'center':'BANKRUPTCY_INDEX_DATE'}>,
        <AxesSubplot:title={'center':'HAS_DIRECT_DEP'}>],
       [<AxesSubplot:title={'center':'CALC_CURRENT_MONTH_END_SHARE_BALANCE'}>,
        <AxesSubplot:title={'center':'NEG_SHARE_BALANCE'}>,
        <AxesSubplot:title={'center':'LOYALTY_DUMMY_MEMBER'}>],
       [<AxesSubplot:title={'center':'CALC_PRIOR_MONTH_END_SHARE_BALANCE'}>,
        <AxesSubplot:title={'center':'OPEN_DATE'}>,
        <AxesSubplot:title={'center':'INT_ACCRUED'}>],
       [<AxesSubplot:title={'center':'LGD%'}>,
        <AxesSubplot:title={'center':'new_england'}>, <AxesSubplot:>]],
      dtype=object)
```


LoanIDNumber
LOAN_NBR
Charge-Off
Recovery
NCO
FIRST_CO_Month_End_Date
MONTH_END_DATE
BALANCE
LIMIT
LOAN_TYPE
PRODUCT_NAME
TIMES_DELQ_30_DAYS
TIMES_DELQ_60_DAYS
TIMES_DELQ_90_DAYS
CREDIT_SCORE
CREDIT_SCORE_DATE
MONTHS_OLD
SEASONED_OVER_2YR
BANKPRUPTCY_FLAG
LOYALTY_LEVEL_ID
MEMBER_TYPE



Importing various packages

In [ ]:
```
import statsmodels.api as sm
from statsmodels.stats.outliers_influence \
import variance_inflation_factor as VIF
from statsmodels.stats.anova import anova_lm
import ISLP
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
summarize ,
poly)
```

In [ ]:
```
df.columns
```

Out[ ]:
```
Index(['LoanIDNumber', 'LOAN_NBR', 'Charge-Off', 'Recovery', 'NCO',
       'FIRST_CO_Month_End_Date', 'MONTH_END_DATE', 'BALANCE', 'LIMIT',
       'LOAN_TYPE', 'PRODUCT_NAME', 'TIMES_DELQ_30_DAYS', 'TIMES_DELQ_60_DAYS',
       'TIMES_DELQ_90_DAYS', 'CREDIT_SCORE', 'CREDIT_SCORE_DATE',
       'CALC_LOANSTATUS', 'MONTHS_OLD', 'SEASONED_OVER_2YR',
       'BANKPRUPTCY_FLAG', 'LOYALTY_LEVEL_ID', 'MEMBER_TYPE',
       'BANKRUPTCY_INDEX', 'BANKRUPTCY_INDEX_DATE', 'HAS_DIRECT_DEP',
       'CALC_CURRENT_MONTH_END_SHARE_BALANCE', 'NEG_SHARE_BALANCE',
       'LOYALTY_DUMMY_MEMBER', 'CALC_PRIOR_MONTH_END_SHARE_BALANCE',
       'OPEN_DATE', 'INT_ACCRUED', 'LGD%', 'new_england'],
      dtype='object')
```

In [ ]:
```python
data_types = df.dtypes
print(data_types)
```

```
LoanIDNumber                                     int64
LOAN_NBR                                         int64
Charge-Off                                     float64
Recovery                                       float64
NCO                                            float64
FIRST_CO_Month_End_Date                 datetime64[ns]
MONTH_END_DATE                          datetime64[ns]
BALANCE                                        float64
LIMIT                                          float64
LOAN_TYPE                                        int32
PRODUCT_NAME                                     int64
TIMES_DELQ_30_DAYS                               int32
TIMES_DELQ_60_DAYS                               int32
TIMES_DELQ_90_DAYS                               int32
CREDIT_SCORE                                     int32
CREDIT_SCORE_DATE                       datetime64[ns]
CALC_LOANSTATUS                                 object
MONTHS_OLD                                       int32
SEASONED_OVER_2YR                                int32
BANKPRUPTCY_FLAG                                 int32
LOYALTY_LEVEL_ID                                 int32
MEMBER_TYPE                                      int64
BANKRUPTCY_INDEX                                 int32
BANKRUPTCY_INDEX_DATE                   datetime64[ns]
HAS_DIRECT_DEP                                   int32
CALC_CURRENT_MONTH_END_SHARE_BALANCE           float64
NEG_SHARE_BALANCE                                int32
LOYALTY_DUMMY_MEMBER                             int32
CALC_PRIOR_MONTH_END_SHARE_BALANCE             float64
OPEN_DATE                               datetime64[ns]
INT_ACCRUED                                    float64
LGD%                                           float64
new_england                                      int64
dtype: object
```

Creating new columns to show differences between the 2 dates in the data provided (just to find a corelation)

In [ ]:
```python
# Create new columns for the differences from FIRST_CO_MONTH_END
df['Diff_FIRST_CO_MONTH_END'] = df['FIRST_CO_Month_End_Date'] - df['MONTH_END_DATE']
df['Diff_FIRST_CO_CREDIT_SCORE'] = df['FIRST_CO_Month_End_Date'] - df['CREDIT_SCORE_DA
df['Diff_FIRST_CO_BANKRUPTCY'] = df['FIRST_CO_Month_End_Date'] - df['BANKRUPTCY_INDEX_
# Create new columns for the differences from MONTH_END_DATE
df['Diff_MONTH_END_CREDIT_SCORE'] = df['MONTH_END_DATE'] - df['CREDIT_SCORE_DATE']
```

```
df['Diff_MONTH_END_BANKRUPTCY'] = df['MONTH_END_DATE'] - df['BANKRUPTCY_INDEX_DATE']
# Create new columns for the differences from OPEN_DATE
df['Diff_OPEN_DATE_FIRST_CO'] = df['FIRST_CO_Month_End_Date'] - df['OPEN_DATE']
df['Diff_OPEN_DATE_MONTH_END'] = df['MONTH_END_DATE'] - df['OPEN_DATE']
df['Diff_OPEN_DATE_CREDIT_SCORE'] = df['CREDIT_SCORE_DATE'] - df['OPEN_DATE']
df['Diff_OPEN_DATE_BANKRUPTCY'] = df['BANKRUPTCY_INDEX_DATE'] - df['OPEN_DATE']

# Convert them into days
df['Diff_FIRST_CO_MONTH_END'] = df['Diff_FIRST_CO_MONTH_END'].dt.days
df['Diff_FIRST_CO_CREDIT_SCORE'] = df['Diff_FIRST_CO_CREDIT_SCORE'].dt.days
df['Diff_FIRST_CO_BANKRUPTCY'] = df['Diff_FIRST_CO_BANKRUPTCY'].dt.days
df['Diff_MONTH_END_CREDIT_SCORE'] = df['Diff_MONTH_END_CREDIT_SCORE'].dt.days
df['Diff_MONTH_END_BANKRUPTCY'] = df['Diff_MONTH_END_BANKRUPTCY'].dt.days
df['Diff_OPEN_DATE_FIRST_CO'] = df['Diff_OPEN_DATE_FIRST_CO'].dt.days
df['Diff_OPEN_DATE_MONTH_END'] = df['Diff_OPEN_DATE_MONTH_END'].dt.days
df['Diff_OPEN_DATE_CREDIT_SCORE'] = df['Diff_OPEN_DATE_CREDIT_SCORE'].dt.days
df['Diff_OPEN_DATE_BANKRUPTCY'] = df['Diff_OPEN_DATE_BANKRUPTCY'].dt.days
```

Dropping Date columns to avoid date-time format

In [ ]:
```
columns_to_drop = ['FIRST_CO_Month_End_Date', 'CREDIT_SCORE_DATE', 'BANKRUPTCY_INDEX_D

df.drop(columns=columns_to_drop, inplace=True)
```

In [ ]:
```
# drop CALC_LOANSTATUS

df.drop(columns='CALC_LOANSTATUS', inplace=True)
```

In [ ]:
```
print(df[['Diff_FIRST_CO_MONTH_END', 'Diff_FIRST_CO_CREDIT_SCORE', 'Diff_FIRST_CO_BANK
```

```
       Diff_FIRST_CO_MONTH_END  Diff_FIRST_CO_CREDIT_SCORE  \
0                          153                         215
1                          153                         242
2                          304                         337
3                          242                         248
4                          151                         215
...                        ...                         ...
29796                      212                         243
29797                      245                         364
29798                      212                         243
29799                      365                         396
29800                      395                         518

       Diff_FIRST_CO_BANKRUPTCY  Diff_MONTH_END_CREDIT_SCORE  \
0                           215                           62
1                           242                           89
2                           337                           33
3                           273                            6
4                           215                           64
...                         ...                          ...
29796                       243                           31
29797                       364                          119
29798                       243                           31
29799                       396                           31
29800                       518                          123

       Diff_MONTH_END_BANKRUPTCY  Diff_OPEN_DATE_MONTH_END  \
0                             62                      5790
1                             89                      5816
2                             33                      4813
3                             31                      2398
4                             64                      2138
...                          ...                       ...
29796                         31                      8284
29797                        119                      6683
29798                         31                      3135
29799                         31                     10195
29800                        123                      1047

       Diff_OPEN_DATE_CREDIT_SCORE  Diff_OPEN_DATE_BANKRUPTCY
0                             5728                       5728
1                             5727                       5727
2                             4780                       4780
3                             2392                       2367
4                             2074                       2074
...                            ...                        ...
29796                         8253                       8253
29797                         6564                       6564
29798                         3104                       3104
29799                        10164                      10164
29800                          924                        924

[22168 rows x 8 columns]
```

In [ ]:
```
print(df[['Diff_OPEN_DATE_MONTH_END','Diff_OPEN_DATE_FIRST_CO', 'Diff_OPEN_DATE_CREDIT
```

```
       Diff_OPEN_DATE_MONTH_END  Diff_OPEN_DATE_FIRST_CO  \
0                          5790                     5943
1                          5816                     5969
2                          4813                     5117
3                          2398                     2640
4                          2138                     2289
...                         ...                      ...
29796                      8284                     8496
29797                      6683                     6928
29798                      3135                     3347
29799                     10195                    10560
29800                      1047                     1442

       Diff_OPEN_DATE_CREDIT_SCORE  Diff_OPEN_DATE_BANKRUPTCY
0                             5728                       5728
1                             5727                       5727
2                             4780                       4780
3                             2392                       2367
4                             2074                       2074
...                            ...                        ...
29796                         8253                       8253
29797                         6564                       6564
29798                         3104                       3104
29799                        10164                      10164
29800                          924                        924

[22168 rows x 4 columns]
```

In [ ]:
```
#Created df_trial to show the stepwise VIF in this
df_trial = df.copy()

# Drop 'MONTH_END_DATE' column from df3
df_trial = df_trial.drop('MONTH_END_DATE', axis=1)
```

In [ ]:
```
terms = df_trial.columns.drop('Recovery')
print(terms)
```

```
Index(['LoanIDNumber', 'LOAN_NBR', 'Charge-Off', 'NCO', 'BALANCE', 'LIMIT',
       'LOAN_TYPE', 'PRODUCT_NAME', 'TIMES_DELQ_30_DAYS', 'TIMES_DELQ_60_DAYS',
       'TIMES_DELQ_90_DAYS', 'CREDIT_SCORE', 'MONTHS_OLD', 'SEASONED_OVER_2YR',
       'BANKPRUPTCY_FLAG', 'LOYALTY_LEVEL_ID', 'MEMBER_TYPE',
       'BANKRUPTCY_INDEX', 'HAS_DIRECT_DEP',
       'CALC_CURRENT_MONTH_END_SHARE_BALANCE', 'NEG_SHARE_BALANCE',
       'LOYALTY_DUMMY_MEMBER', 'CALC_PRIOR_MONTH_END_SHARE_BALANCE',
       'INT_ACCRUED', 'LGD%', 'new_england', 'Diff_FIRST_CO_MONTH_END',
       'Diff_FIRST_CO_CREDIT_SCORE', 'Diff_FIRST_CO_BANKRUPTCY',
       'Diff_MONTH_END_CREDIT_SCORE', 'Diff_MONTH_END_BANKRUPTCY',
       'Diff_OPEN_DATE_FIRST_CO', 'Diff_OPEN_DATE_MONTH_END',
       'Diff_OPEN_DATE_CREDIT_SCORE', 'Diff_OPEN_DATE_BANKRUPTCY'],
      dtype='object')
```

In [ ]:
```
column_names = df_trial.columns
print(column_names)
```

```
Index(['LoanIDNumber', 'LOAN_NBR', 'Charge-Off', 'Recovery', 'NCO', 'BALANCE',
       'LIMIT', 'LOAN_TYPE', 'PRODUCT_NAME', 'TIMES_DELQ_30_DAYS',
       'TIMES_DELQ_60_DAYS', 'TIMES_DELQ_90_DAYS', 'CREDIT_SCORE',
       'MONTHS_OLD', 'SEASONED_OVER_2YR', 'BANKPRUPTCY_FLAG',
       'LOYALTY_LEVEL_ID', 'MEMBER_TYPE', 'BANKRUPTCY_INDEX', 'HAS_DIRECT_DEP',
       'CALC_CURRENT_MONTH_END_SHARE_BALANCE', 'NEG_SHARE_BALANCE',
       'LOYALTY_DUMMY_MEMBER', 'CALC_PRIOR_MONTH_END_SHARE_BALANCE',
       'INT_ACCRUED', 'LGD%', 'new_england', 'Diff_FIRST_CO_MONTH_END',
       'Diff_FIRST_CO_CREDIT_SCORE', 'Diff_FIRST_CO_BANKRUPTCY',
       'Diff_MONTH_END_CREDIT_SCORE', 'Diff_MONTH_END_BANKRUPTCY',
       'Diff_OPEN_DATE_FIRST_CO', 'Diff_OPEN_DATE_MONTH_END',
       'Diff_OPEN_DATE_CREDIT_SCORE', 'Diff_OPEN_DATE_BANKRUPTCY'],
      dtype='object')
```

In [ ]:
```
print(df_trial.dtypes)
```

```
LoanIDNumber                              int64
LOAN_NBR                                  int64
Charge-Off                              float64
Recovery                                float64
NCO                                     float64
BALANCE                                 float64
LIMIT                                   float64
LOAN_TYPE                                 int32
PRODUCT_NAME                              int64
TIMES_DELQ_30_DAYS                        int32
TIMES_DELQ_60_DAYS                        int32
TIMES_DELQ_90_DAYS                        int32
CREDIT_SCORE                              int32
MONTHS_OLD                                int32
SEASONED_OVER_2YR                         int32
BANKPRUPTCY_FLAG                          int32
LOYALTY_LEVEL_ID                          int32
MEMBER_TYPE                               int64
BANKRUPTCY_INDEX                          int32
HAS_DIRECT_DEP                            int32
CALC_CURRENT_MONTH_END_SHARE_BALANCE    float64
NEG_SHARE_BALANCE                         int32
LOYALTY_DUMMY_MEMBER                      int32
CALC_PRIOR_MONTH_END_SHARE_BALANCE      float64
INT_ACCRUED                             float64
LGD%                                    float64
new_england                               int64
Diff_FIRST_CO_MONTH_END                   int64
Diff_FIRST_CO_CREDIT_SCORE                int64
Diff_FIRST_CO_BANKRUPTCY                  int64
Diff_MONTH_END_CREDIT_SCORE               int64
Diff_MONTH_END_BANKRUPTCY                 int64
Diff_OPEN_DATE_FIRST_CO                   int64
Diff_OPEN_DATE_MONTH_END                  int64
Diff_OPEN_DATE_CREDIT_SCORE               int64
Diff_OPEN_DATE_BANKRUPTCY                 int64
dtype: object
```

Split the data into train and test sets on the basis of years in MONTH_END_DATE

In [ ]:
```
import pandas as pd


train_set = df.loc[(df['MONTH_END_DATE'] >= '2013-01-01') & (df['MONTH_END_DATE'] <= '
```

```
test_set_1 = df.loc[(df['MONTH_END_DATE'] >= '2018-01-01') & (df['MONTH_END_DATE'] <=
test_set_2 = df.loc[(df['MONTH_END_DATE'] >= '2020-04-01') & (df['MONTH_END_DATE'] <=
test_set_3 = df.loc[(df['MONTH_END_DATE'] >= '2021-01-01') & (df['MONTH_END_DATE'] <=
test_set_4 = df.loc[(df['MONTH_END_DATE'] >= '2022-01-01') & (df['MONTH_END_DATE'] <=


print("Train Set Shape:", train_set.shape)
print("Test Set 1 Shape:", test_set_1.shape)
print("Test Set 2 Shape:", test_set_2.shape)
print("Test Set 3 Shape:", test_set_3.shape)
print("Test Set 4 Shape:", test_set_4.shape)
```

```
Train Set Shape: (8730, 37)
Test Set 1 Shape: (8313, 37)
Test Set 2 Shape: (1504, 37)
Test Set 3 Shape: (2416, 37)
Test Set 4 Shape: (667, 37)
```

In [ ]:
```
print("Columns of Train Set:", train_set.columns)
```

```
Columns of Train Set: Index(['LoanIDNumber', 'LOAN_NBR', 'Charge-Off', 'Recovery', 'N
CO',
       'MONTH_END_DATE', 'BALANCE', 'LIMIT', 'LOAN_TYPE', 'PRODUCT_NAME',
       'TIMES_DELQ_30_DAYS', 'TIMES_DELQ_60_DAYS', 'TIMES_DELQ_90_DAYS',
       'CREDIT_SCORE', 'MONTHS_OLD', 'SEASONED_OVER_2YR', 'BANKPRUPTCY_FLAG',
       'LOYALTY_LEVEL_ID', 'MEMBER_TYPE', 'BANKRUPTCY_INDEX', 'HAS_DIRECT_DEP',
       'CALC_CURRENT_MONTH_END_SHARE_BALANCE', 'NEG_SHARE_BALANCE',
       'LOYALTY_DUMMY_MEMBER', 'CALC_PRIOR_MONTH_END_SHARE_BALANCE',
       'INT_ACCRUED', 'LGD%', 'new_england', 'Diff_FIRST_CO_MONTH_END',
       'Diff_FIRST_CO_CREDIT_SCORE', 'Diff_FIRST_CO_BANKRUPTCY',
       'Diff_MONTH_END_CREDIT_SCORE', 'Diff_MONTH_END_BANKRUPTCY',
       'Diff_OPEN_DATE_FIRST_CO', 'Diff_OPEN_DATE_MONTH_END',
       'Diff_OPEN_DATE_CREDIT_SCORE', 'Diff_OPEN_DATE_BANKRUPTCY'],
      dtype='object')
```

Drop MED to remove the date-time format

In [ ]:
```
# Drop 'MONTH_END_DATE' column from train_set
train_set = train_set.drop('MONTH_END_DATE', axis=1)

# Drop 'MONTH_END_DATE' column from test_set_1
test_set_1 = test_set_1.drop('MONTH_END_DATE', axis=1)

# Drop 'MONTH_END_DATE' column from test_set_2
test_set_2 = test_set_2.drop('MONTH_END_DATE', axis=1)

# Drop 'MONTH_END_DATE' column from test_set_3
test_set_3 = test_set_3.drop('MONTH_END_DATE', axis=1)

# Drop 'MONTH_END_DATE' column from test_set_4
test_set_4 = test_set_4.drop('MONTH_END_DATE', axis=1)
```

In [ ]:
```
print("Columns of Train Set:", train_set.columns)
```

```
Columns of Train Set: Index(['LoanIDNumber', 'LOAN_NBR', 'Charge-Off', 'Recovery', 'N
CO', 'BALANCE',
       'LIMIT', 'LOAN_TYPE', 'PRODUCT_NAME', 'TIMES_DELQ_30_DAYS',
       'TIMES_DELQ_60_DAYS', 'TIMES_DELQ_90_DAYS', 'CREDIT_SCORE',
       'MONTHS_OLD', 'SEASONED_OVER_2YR', 'BANKPRUPTCY_FLAG',
       'LOYALTY_LEVEL_ID', 'MEMBER_TYPE', 'BANKRUPTCY_INDEX', 'HAS_DIRECT_DEP',
       'CALC_CURRENT_MONTH_END_SHARE_BALANCE', 'NEG_SHARE_BALANCE',
       'LOYALTY_DUMMY_MEMBER', 'CALC_PRIOR_MONTH_END_SHARE_BALANCE',
       'INT_ACCRUED', 'LGD%', 'new_england', 'Diff_FIRST_CO_MONTH_END',
       'Diff_FIRST_CO_CREDIT_SCORE', 'Diff_FIRST_CO_BANKRUPTCY',
       'Diff_MONTH_END_CREDIT_SCORE', 'Diff_MONTH_END_BANKRUPTCY',
       'Diff_OPEN_DATE_FIRST_CO', 'Diff_OPEN_DATE_MONTH_END',
       'Diff_OPEN_DATE_CREDIT_SCORE', 'Diff_OPEN_DATE_BANKRUPTCY'],
      dtype='object')
```

VIF: Variance Inflation Factor

VIF is a statistical measure in regression analysis used to assess multicollinearity among predictor variables. Multicollinearity arises when two or more independent variables are highly correlated, impacting the interpretation and reliability of regression results.

In our analysis, we retained variables with a VIF below 5, excluding Credit Score, which exhibited higher VIF.

In [ ]:
```
X = MS(terms).fit_transform(df_trial)
y = df_trial['Recovery']
model = sm.OLS(y, X)
results = model.fit()
summarize(results)
```

Out[ ]:

	coef	std err	t	P>|t|
intercept	9.359000e-14	5.450000e-15	1.716900e+01	0.000
LoanIDNumber	-2.108000e-16	6.140000e-18	-3.430500e+01	0.000
LOAN_NBR	-1.259000e-12	5.250000e-14	-2.398800e+01	0.000
Charge-Off	1.000000e+00	1.020000e-16	9.780000e+15	0.000
NCO	-1.000000e+00	3.670000e-17	-2.720000e+16	0.000
BALANCE	2.082000e-15	9.300000e-17	2.237500e+01	0.000
LIMIT	8.804000e-17	1.350000e-17	6.516000e+00	0.000
LOAN_TYPE	2.747000e-12	1.130000e-13	2.428800e+01	0.000
PRODUCT_NAME	-3.482000e-12	3.010000e-13	-1.155300e+01	0.000
TIMES_DELQ_30_DAYS	-1.310000e-14	2.620000e-14	-5.000000e-01	0.617
TIMES_DELQ_60_DAYS	5.329000e-14	7.870000e-14	6.780000e-01	0.498
TIMES_DELQ_90_DAYS	2.998000e-14	8.080000e-14	3.710000e-01	0.711
CREDIT_SCORE	5.395000e-16	6.830000e-16	7.900000e-01	0.430
MONTHS_OLD	-1.843000e-14	1.510000e-13	-1.220000e-01	0.903
SEASONED_OVER_2YR	3.286000e-14	1.150000e-13	2.860000e-01	0.775
BANKPRUPTCY_FLAG	8.882000e-14	3.340000e-13	2.660000e-01	0.790
LOYALTY_LEVEL_ID	9.148000e-14	1.450000e-13	6.320000e-01	0.527
MEMBER_TYPE	-8.792000e-21	1.170000e-28	-7.530000e+07	0.000
BANKRUPTCY_INDEX	-4.129000e-16	5.590000e-16	-7.390000e-01	0.460
HAS_DIRECT_DEP	2.265000e-13	1.330000e-13	1.709000e+00	0.088
CALC_CURRENT_MONTH_END_SHARE_BALANCE	-5.330000e-16	1.510000e-17	-3.529800e+01	0.000
NEG_SHARE_BALANCE	-3.829000e-13	1.560000e-13	-2.450000e+00	0.014
LOYALTY_DUMMY_MEMBER	9.059000e-14	2.350000e-13	3.860000e-01	0.700
CALC_PRIOR_MONTH_END_SHARE_BALANCE	3.204000e-16	1.270000e-17	2.530100e+01	0.000
INT_ACCRUED	1.190000e-15	5.430000e-16	2.192000e+00	0.028
LGD%	3.411000e-13	7.590000e-13	4.490000e-01	0.653
new_england	1.039000e-13	9.870000e-14	1.053000e+00	0.293
Diff_FIRST_CO_MONTH_END	-2.949000e-17	1.010000e-15	-2.900000e-02	0.977
Diff_FIRST_CO_CREDIT_SCORE	3.244000e-16	3.890000e-16	8.340000e-01	0.404
Diff_FIRST_CO_BANKRUPTCY	1.610000e-15	3.920000e-16	4.106000e+00	0.000
Diff_MONTH_END_CREDIT_SCORE	1.900000e-16	1.070000e-15	1.770000e-01	0.859
Diff_MONTH_END_BANKRUPTCY	1.315000e-15	1.080000e-15	1.220000e+00	0.222
Diff_OPEN_DATE_FIRST_CO	2.359000e-16	9.930000e-16	2.380000e-01	0.812
Diff_OPEN_DATE_MONTH_END	4.233000e-16	1.990000e-15	2.130000e-01	0.831

	coef	std err	t	P>|t|
Diff_OPEN_DATE_CREDIT_SCORE	8.760000e-16	1.060000e-15	8.240000e-01	0.410
Diff_OPEN_DATE_BANKRUPTCY	-7.286000e-16	1.060000e-15	-6.860000e-01	0.493

In [ ]:
```
vals = [VIF(X, i)
for i in range(1, X.shape[1])]
vif = pd.DataFrame({'vif':vals},
index=X.columns [1:])
vif
```

```
c:\Users\19783\anaconda3\lib\site-packages\statsmodels\stats\outliers_influence.py:19
5: RuntimeWarning: divide by zero encountered in scalar divide
  vif = 1. / (1. - r_squared_i)
c:\Users\19783\anaconda3\lib\site-packages\statsmodels\regression\linear_model.py:173
6: RuntimeWarning: invalid value encountered in scalar divide
  return 1 - self.ssr/self.centered_tss
```

Out[ ]:

	vif
LoanIDNumber	1.504425e+00
LOAN_NBR	1.058082e+00
Charge-Off	1.173096e+02
NCO	1.507364e+01
BALANCE	9.868863e+01
LIMIT	2.142900e+00
LOAN_TYPE	inf
PRODUCT_NAME	inf
TIMES_DELQ_30_DAYS	6.103153e+00
TIMES_DELQ_60_DAYS	1.759135e+01
TIMES_DELQ_90_DAYS	8.210234e+00
CREDIT_SCORE	1.888964e+00
MONTHS_OLD	4.448981e+04
SEASONED_OVER_2YR	1.696106e+00
BANKPRUPTCY_FLAG	1.046513e+00
LOYALTY_LEVEL_ID	8.926528e+00
MEMBER_TYPE	NaN
BANKRUPTCY_INDEX	1.801810e+00
HAS_DIRECT_DEP	2.106838e+00
CALC_CURRENT_MONTH_END_SHARE_BALANCE	3.337170e+00
NEG_SHARE_BALANCE	1.067774e+00
LOYALTY_DUMMY_MEMBER	7.337803e+00
CALC_PRIOR_MONTH_END_SHARE_BALANCE	3.350566e+00
INT_ACCRUED	1.466328e+00
LGD%	1.931187e+00
new_england	1.207995e+00
Diff_FIRST_CO_MONTH_END	inf
Diff_FIRST_CO_CREDIT_SCORE	inf
Diff_FIRST_CO_BANKRUPTCY	inf
Diff_MONTH_END_CREDIT_SCORE	inf
Diff_MONTH_END_BANKRUPTCY	inf
Diff_OPEN_DATE_FIRST_CO	inf
Diff_OPEN_DATE_MONTH_END	inf
Diff_OPEN_DATE_CREDIT_SCORE	inf

	vif
Diff_OPEN_DATE_BANKRUPTCY	inf

In [ ]:
```
print(df)
```

```
       LoanIDNumber  LOAN_NBR  Charge-Off  Recovery       NCO MONTH_END_DATE  \
0             39035       141    12649.67       0.0  12649.67     2014-08-31
1             39056       141     8198.21       0.0   8198.21     2013-03-31
2             39057       142     5779.70       0.0   5779.70     2013-04-30
3             39058       143     1995.31       0.0   1995.31     2021-01-31
4             39059       143     1818.41       0.0   1818.41     2018-11-30
...             ...       ...         ...       ...       ...            ...
29796         39731       141     3286.22       0.0   3286.22     2021-01-31
29797         10334       142     9928.17       0.0   9928.17     2018-04-30
29798         10335       141     9976.79       0.0   9976.79     2016-07-31
29799         10336       141    17092.65       0.0  17092.65     2021-10-31
29800         39732       142     1500.00       0.0   1500.00     2017-10-31

        BALANCE    LIMIT  LOAN_TYPE  PRODUCT_NAME  ...  new_england  \
0      12649.67  14000.0         66             0  ...            0
1       8198.21      0.0         66             0  ...            1
2       5779.70   7000.0         66             0  ...            0
3       1995.31  10000.0         66             0  ...            1
4       1818.41      0.0         66             0  ...            0
...         ...      ...        ...           ...  ...          ...
29796   3286.22  17000.0         66             0  ...            1
29797   9928.17      0.0         66             0  ...            1
29798   9976.79  10000.0         66             0  ...            0
29799  17325.51  27000.0         66             0  ...            0
29800   1500.00   1500.0         66             0  ...            0

       Diff_FIRST_CO_MONTH_END  Diff_FIRST_CO_CREDIT_SCORE  \
0                          153                         215
1                          153                         242
2                          304                         337
3                          242                         248
4                          151                         215
...                        ...                         ...
29796                      212                         243
29797                      245                         364
29798                      212                         243
29799                      365                         396
29800                      395                         518

       Diff_FIRST_CO_BANKRUPTCY  Diff_MONTH_END_CREDIT_SCORE  \
0                           215                           62
1                           242                           89
2                           337                           33
3                           273                            6
4                           215                           64
...                         ...                          ...
29796                       243                           31
29797                       364                          119
29798                       243                           31
29799                       396                           31
29800                       518                          123

       Diff_MONTH_END_BANKRUPTCY  Diff_OPEN_DATE_FIRST_CO  \
0                             62                     5943
1                             89                     5969
2                             33                     5117
3                             31                     2640
4                             64                     2289
...                          ...                      ...
29796                         31                     8496
```

```
29797                        119                      6928
29798                         31                      3347
29799                         31                     10560
29800                        123                      1442

       Diff_OPEN_DATE_MONTH_END  Diff_OPEN_DATE_CREDIT_SCORE  \
0                          5790                         5728
1                          5816                         5727
2                          4813                         4780
3                          2398                         2392
4                          2138                         2074
...                         ...                          ...
29796                      8284                         8253
29797                      6683                         6564
29798                      3135                         3104
29799                     10195                        10164
29800                      1047                          924

       Diff_OPEN_DATE_BANKRUPTCY
0                           5728
1                           5727
2                           4780
3                           2367
4                           2074
...                          ...
29796                       8253
29797                       6564
29798                       3104
29799                      10164
29800                        924

[22168 rows x 37 columns]
```

In [ ]:

```
def vif(dfx):
    vals = [VIF(dfx, i) for i in range(1, dfx.shape[1])]
    vif_var = pd.DataFrame({'vif': vals}, index=dfx.columns[1:])
    return vif_var
```

"Recovery" is the dependent variable, and we are excluding it out of our dataframe

In [ ]:

```
independent_variables = df_trial.drop(columns=['Recovery'])

result = vif(independent_variables)
print(result)
```

```
c:\Users\19783\anaconda3\lib\site-packages\statsmodels\regression\linear_model.py:173
6: RuntimeWarning: invalid value encountered in scalar divide
  return 1 - self.ssr/self.centered_tss
c:\Users\19783\anaconda3\lib\site-packages\statsmodels\stats\outliers_influence.py:19
5: RuntimeWarning: divide by zero encountered in scalar divide
  vif = 1. / (1. - r_squared_i)
```

```
                                          vif
LOAN_NBR                         1.058082e+00
Charge-Off                       1.173096e+02
NCO                              1.507364e+01
BALANCE                          9.868863e+01
LIMIT                            2.142900e+00
LOAN_TYPE                        2.972230e+04
PRODUCT_NAME                     1.241911e+00
TIMES_DELQ_30_DAYS               6.103153e+00
TIMES_DELQ_60_DAYS               1.759135e+01
TIMES_DELQ_90_DAYS               8.210234e+00
CREDIT_SCORE                     1.888964e+00
MONTHS_OLD                       4.448981e+04
SEASONED_OVER_2YR                1.696106e+00
BANKPRUPTCY_FLAG                 1.046513e+00
LOYALTY_LEVEL_ID                 8.926528e+00
MEMBER_TYPE                               NaN
BANKRUPTCY_INDEX                 1.801810e+00
HAS_DIRECT_DEP                   2.106838e+00
CALC_CURRENT_MONTH_END_SHARE_BALANCE  3.337170e+00
NEG_SHARE_BALANCE                1.067774e+00
LOYALTY_DUMMY_MEMBER             7.337803e+00
CALC_PRIOR_MONTH_END_SHARE_BALANCE    3.350566e+00
INT_ACCRUED                      1.466328e+00
LGD%                             1.931187e+00
new_england                      1.207995e+00
Diff_FIRST_CO_MONTH_END                   inf
Diff_FIRST_CO_CREDIT_SCORE                inf
Diff_FIRST_CO_BANKRUPTCY                  inf
Diff_MONTH_END_CREDIT_SCORE               inf
Diff_MONTH_END_BANKRUPTCY                 inf
Diff_OPEN_DATE_FIRST_CO                   inf
Diff_OPEN_DATE_MONTH_END                  inf
Diff_OPEN_DATE_CREDIT_SCORE               inf
Diff_OPEN_DATE_BANKRUPTCY                 inf
```

In [ ]:

```
# Try : Removing MEMBER_TYPE since it has Nan

independent_variables1 = independent_variables.drop(columns=['MEMBER_TYPE'])
result = vif(independent_variables1)
print(result)
```

```
c:\Users\19783\anaconda3\lib\site-packages\statsmodels\stats\outliers_influence.py:19
5: RuntimeWarning: divide by zero encountered in scalar divide
  vif = 1. / (1. - r_squared_i)
```

```
                                                   vif
LOAN_NBR                                  1.058082e+00
Charge-Off                                1.173096e+02
NCO                                       1.507364e+01
BALANCE                                   9.868863e+01
LIMIT                                     2.142900e+00
LOAN_TYPE                                 2.972230e+04
PRODUCT_NAME                              1.241911e+00
TIMES_DELQ_30_DAYS                        6.103153e+00
TIMES_DELQ_60_DAYS                        1.759135e+01
TIMES_DELQ_90_DAYS                        8.210234e+00
CREDIT_SCORE                              1.888964e+00
MONTHS_OLD                                4.448981e+04
SEASONED_OVER_2YR                         1.696106e+00
BANKPRUPTCY_FLAG                          1.046513e+00
LOYALTY_LEVEL_ID                          8.926528e+00
BANKRUPTCY_INDEX                          1.801810e+00
HAS_DIRECT_DEP                            2.106838e+00
CALC_CURRENT_MONTH_END_SHARE_BALANCE      3.337170e+00
NEG_SHARE_BALANCE                         1.067774e+00
LOYALTY_DUMMY_MEMBER                      7.337803e+00
CALC_PRIOR_MONTH_END_SHARE_BALANCE        3.350566e+00
INT_ACCRUED                               1.466328e+00
LGD%                                      1.931187e+00
new_england                               1.207995e+00
Diff_FIRST_CO_MONTH_END                            inf
Diff_FIRST_CO_CREDIT_SCORE                         inf
Diff_FIRST_CO_BANKRUPTCY                           inf
Diff_MONTH_END_CREDIT_SCORE                        inf
Diff_MONTH_END_BANKRUPTCY                          inf
Diff_OPEN_DATE_FIRST_CO                            inf
Diff_OPEN_DATE_MONTH_END                           inf
Diff_OPEN_DATE_CREDIT_SCORE                        inf
Diff_OPEN_DATE_BANKRUPTCY                          inf
```

In [ ]:

```
# Try 1: Removing Diff_FIRST_CO_MONTH_END


independent_variables2 = independent_variables1.drop(columns=['Diff_FIRST_CO_MONTH_END

result = vif(independent_variables2)
print(result)
```

```
c:\Users\19783\anaconda3\lib\site-packages\statsmodels\stats\outliers_influence.py:19
5: RuntimeWarning: divide by zero encountered in scalar divide
  vif = 1. / (1. - r_squared_i)
```

```
                                                  vif
LOAN_NBR                                 1.058082e+00
Charge-Off                               1.173096e+02
NCO                                      1.507364e+01
BALANCE                                  9.868863e+01
LIMIT                                    2.142900e+00
LOAN_TYPE                                2.972230e+04
PRODUCT_NAME                             1.241911e+00
TIMES_DELQ_30_DAYS                       6.103153e+00
TIMES_DELQ_60_DAYS                       1.759135e+01
TIMES_DELQ_90_DAYS                       8.210234e+00
CREDIT_SCORE                             1.888964e+00
MONTHS_OLD                               4.448981e+04
SEASONED_OVER_2YR                        1.696106e+00
BANKPRUPTCY_FLAG                         1.046513e+00
LOYALTY_LEVEL_ID                         8.926528e+00
BANKRUPTCY_INDEX                         1.801810e+00
HAS_DIRECT_DEP                           2.106838e+00
CALC_CURRENT_MONTH_END_SHARE_BALANCE     3.337170e+00
NEG_SHARE_BALANCE                        1.067774e+00
LOYALTY_DUMMY_MEMBER                     7.337803e+00
CALC_PRIOR_MONTH_END_SHARE_BALANCE       3.350566e+00
INT_ACCRUED                              1.466328e+00
LGD%                                     1.931187e+00
new_england                              1.207995e+00
Diff_FIRST_CO_CREDIT_SCORE                        inf
Diff_FIRST_CO_BANKRUPTCY                          inf
Diff_MONTH_END_CREDIT_SCORE                       inf
Diff_MONTH_END_BANKRUPTCY                         inf
Diff_OPEN_DATE_FIRST_CO                           inf
Diff_OPEN_DATE_MONTH_END                          inf
Diff_OPEN_DATE_CREDIT_SCORE                       inf
Diff_OPEN_DATE_BANKRUPTCY                         inf
```

In [ ]:

```
# Try 2: Removing Diff_FIRST_CO_CREDIT_SCORE

independent_variables3 = independent_variables2.drop(columns=['Diff_FIRST_CO_CREDIT_SC

result = vif(independent_variables3)
print(result)
```

```
c:\Users\19783\anaconda3\lib\site-packages\statsmodels\stats\outliers_influence.py:19
5: RuntimeWarning: divide by zero encountered in scalar divide
  vif = 1. / (1. - r_squared_i)
```

```
                                                  vif
LOAN_NBR                                 1.058082e+00
Charge-Off                               1.173096e+02
NCO                                      1.507364e+01
BALANCE                                  9.868863e+01
LIMIT                                    2.142900e+00
LOAN_TYPE                                2.972230e+04
PRODUCT_NAME                             1.241911e+00
TIMES_DELQ_30_DAYS                       6.103153e+00
TIMES_DELQ_60_DAYS                       1.759135e+01
TIMES_DELQ_90_DAYS                       8.210234e+00
CREDIT_SCORE                             1.888964e+00
MONTHS_OLD                               4.448981e+04
SEASONED_OVER_2YR                        1.696106e+00
BANKPRUPTCY_FLAG                         1.046513e+00
LOYALTY_LEVEL_ID                         8.926528e+00
BANKRUPTCY_INDEX                         1.801810e+00
HAS_DIRECT_DEP                           2.106838e+00
CALC_CURRENT_MONTH_END_SHARE_BALANCE     3.337170e+00
NEG_SHARE_BALANCE                        1.067774e+00
LOYALTY_DUMMY_MEMBER                     7.337803e+00
CALC_PRIOR_MONTH_END_SHARE_BALANCE       3.350566e+00
INT_ACCRUED                              1.466328e+00
LGD%                                     1.931187e+00
new_england                              1.207995e+00
Diff_FIRST_CO_BANKRUPTCY                          inf
Diff_MONTH_END_CREDIT_SCORE                       inf
Diff_MONTH_END_BANKRUPTCY                         inf
Diff_OPEN_DATE_FIRST_CO                           inf
Diff_OPEN_DATE_MONTH_END                          inf
Diff_OPEN_DATE_CREDIT_SCORE                       inf
Diff_OPEN_DATE_BANKRUPTCY                         inf
```

In [ ]:

```
# Try 3: Removing Diff_FIRST_CO_BANKRUPTCY


independent_variables4 = independent_variables3.drop(columns=['Diff_FIRST_CO_BANKRUPTC

result = vif(independent_variables4)
print(result)
```

```
c:\Users\19783\anaconda3\lib\site-packages\statsmodels\stats\outliers_influence.py:19
5: RuntimeWarning: divide by zero encountered in scalar divide
  vif = 1. / (1. - r_squared_i)
```

```
                                                   vif
LOAN_NBR                                  1.058082e+00
Charge-Off                                1.173096e+02
NCO                                       1.507364e+01
BALANCE                                   9.868863e+01
LIMIT                                     2.142900e+00
LOAN_TYPE                                 2.972230e+04
PRODUCT_NAME                              1.241911e+00
TIMES_DELQ_30_DAYS                        6.103153e+00
TIMES_DELQ_60_DAYS                        1.759135e+01
TIMES_DELQ_90_DAYS                        8.210234e+00
CREDIT_SCORE                              1.888964e+00
MONTHS_OLD                                4.448981e+04
SEASONED_OVER_2YR                         1.696106e+00
BANKPRUPTCY_FLAG                          1.046513e+00
LOYALTY_LEVEL_ID                          8.926528e+00
BANKRUPTCY_INDEX                          1.801810e+00
HAS_DIRECT_DEP                            2.106838e+00
CALC_CURRENT_MONTH_END_SHARE_BALANCE      3.337170e+00
NEG_SHARE_BALANCE                         1.067774e+00
LOYALTY_DUMMY_MEMBER                      7.337803e+00
CALC_PRIOR_MONTH_END_SHARE_BALANCE        3.350566e+00
INT_ACCRUED                               1.466328e+00
LGD%                                      1.931187e+00
new_england                               1.207995e+00
Diff_MONTH_END_CREDIT_SCORE                        inf
Diff_MONTH_END_BANKRUPTCY                          inf
Diff_OPEN_DATE_FIRST_CO                   2.449923e+02
Diff_OPEN_DATE_MONTH_END                           inf
Diff_OPEN_DATE_CREDIT_SCORE                        inf
Diff_OPEN_DATE_BANKRUPTCY                          inf
```

In [ ]:

```
# # Try 4: Removing Diff_MONTH_END_CREDIT_SCORE


independent_variables5 = independent_variables4.drop(columns=['Diff_MONTH_END_CREDIT_S

result = vif(independent_variables5)
print(result)
```

```
c:\Users\19783\anaconda3\lib\site-packages\statsmodels\stats\outliers_influence.py:19
5: RuntimeWarning: divide by zero encountered in scalar divide
  vif = 1. / (1. - r_squared_i)
```

```
                                          vif
LOAN_NBR                            1.058082e+00
Charge-Off                          1.173096e+02
NCO                                 1.507364e+01
BALANCE                             9.868863e+01
LIMIT                               2.142900e+00
LOAN_TYPE                           2.972230e+04
PRODUCT_NAME                        1.241911e+00
TIMES_DELQ_30_DAYS                  6.103153e+00
TIMES_DELQ_60_DAYS                  1.759135e+01
TIMES_DELQ_90_DAYS                  8.210234e+00
CREDIT_SCORE                        1.888964e+00
MONTHS_OLD                          4.448981e+04
SEASONED_OVER_2YR                   1.696106e+00
BANKPRUPTCY_FLAG                    1.046513e+00
LOYALTY_LEVEL_ID                    8.926528e+00
BANKRUPTCY_INDEX                    1.801810e+00
HAS_DIRECT_DEP                      2.106838e+00
CALC_CURRENT_MONTH_END_SHARE_BALANCE 3.337170e+00
NEG_SHARE_BALANCE                   1.067774e+00
LOYALTY_DUMMY_MEMBER                7.337803e+00
CALC_PRIOR_MONTH_END_SHARE_BALANCE  3.350566e+00
INT_ACCRUED                         1.466328e+00
LGD%                                1.931187e+00
new_england                         1.207995e+00
Diff_MONTH_END_BANKRUPTCY                    inf
Diff_OPEN_DATE_FIRST_CO             2.449923e+02
Diff_OPEN_DATE_MONTH_END                     inf
Diff_OPEN_DATE_CREDIT_SCORE         2.413937e+03
Diff_OPEN_DATE_BANKRUPTCY                    inf
```

In [ ]:

```
# Try 5: Removing Diff_MONTH_END_BANKRUPTCY

independent_variables6 = independent_variables5.drop(columns=['Diff_MONTH_END_BANKRUPT

result = vif(independent_variables6)
print(result)
```

```
                                          vif
LOAN_NBR                             1.058082
Charge-Off                         117.309603
NCO                                 15.073640
BALANCE                             98.688625
LIMIT                                2.142900
LOAN_TYPE                        29722.303917
PRODUCT_NAME                         1.241911
TIMES_DELQ_30_DAYS                   6.103153
TIMES_DELQ_60_DAYS                  17.591347
TIMES_DELQ_90_DAYS                   8.210234
CREDIT_SCORE                         1.888964
MONTHS_OLD                       44489.813134
SEASONED_OVER_2YR                    1.696106
BANKPRUPTCY_FLAG                     1.046513
LOYALTY_LEVEL_ID                     8.926528
BANKRUPTCY_INDEX                     1.801810
HAS_DIRECT_DEP                       2.106838
CALC_CURRENT_MONTH_END_SHARE_BALANCE 3.337170
NEG_SHARE_BALANCE                    1.067774
LOYALTY_DUMMY_MEMBER                 7.337803
CALC_PRIOR_MONTH_END_SHARE_BALANCE   3.350566
INT_ACCRUED                          1.466328
LGD%                                 1.931187
new_england                          1.207995
Diff_OPEN_DATE_FIRST_CO            244.992278
Diff_OPEN_DATE_MONTH_END         45265.706345
Diff_OPEN_DATE_CREDIT_SCORE       2413.936739
Diff_OPEN_DATE_BANKRUPTCY         2478.865431
```

In [ ]:

```
#Try 6: Removing Diff_OPEN_DATE_MONTH_END

independent_variables7 = independent_variables6.drop(columns=['Diff_OPEN_DATE_MONTH_EN

result = vif(independent_variables7)
print(result)
```

```
                                          vif
LOAN_NBR                             1.058043
Charge-Off                         117.308128
NCO                                 15.073223
BALANCE                             98.687963
LIMIT                                2.142704
LOAN_TYPE                        29722.137179
PRODUCT_NAME                         1.241909
TIMES_DELQ_30_DAYS                   6.102851
TIMES_DELQ_60_DAYS                  17.588297
TIMES_DELQ_90_DAYS                   8.209523
CREDIT_SCORE                         1.888870
MONTHS_OLD                         628.223014
SEASONED_OVER_2YR                    1.695807
BANKPRUPTCY_FLAG                     1.046510
LOYALTY_LEVEL_ID                     8.925053
BANKRUPTCY_INDEX                     1.801794
HAS_DIRECT_DEP                       2.106651
CALC_CURRENT_MONTH_END_SHARE_BALANCE 3.336981
NEG_SHARE_BALANCE                    1.067773
LOYALTY_DUMMY_MEMBER                 7.336915
CALC_PRIOR_MONTH_END_SHARE_BALANCE   3.350550
INT_ACCRUED                          1.466328
LGD%                                 1.931187
new_england                          1.207993
Diff_OPEN_DATE_FIRST_CO            242.671698
Diff_OPEN_DATE_CREDIT_SCORE       2413.532438
Diff_OPEN_DATE_BANKRUPTCY         2477.499367
```

In [ ]:

```
#Try 7: Removing Highest VIF LOAN_TYPE

independent_variables8 = independent_variables7.drop(columns=['LOAN_TYPE'])

result = vif(independent_variables8)
print(result)
```

```
                                          vif
LOAN_NBR                           437.332465
Charge-Off                         220.143909
NCO                                 26.786896
BALANCE                            185.588817
LIMIT                                3.085600
PRODUCT_NAME                         1.063185
TIMES_DELQ_30_DAYS                   7.849282
TIMES_DELQ_60_DAYS                  19.944008
TIMES_DELQ_90_DAYS                   8.760606
CREDIT_SCORE                        87.661023
MONTHS_OLD                        1075.359195
SEASONED_OVER_2YR                    4.123791
BANKPRUPTCY_FLAG                     1.065467
LOYALTY_LEVEL_ID                    16.825512
BANKRUPTCY_INDEX                     9.933770
HAS_DIRECT_DEP                       3.200783
CALC_CURRENT_MONTH_END_SHARE_BALANCE 3.360518
NEG_SHARE_BALANCE                    1.173010
LOYALTY_DUMMY_MEMBER                14.438124
CALC_PRIOR_MONTH_END_SHARE_BALANCE   3.400377
INT_ACCRUED                          2.407918
LGD%                               300.861682
new_england                          1.902463
Diff_OPEN_DATE_FIRST_CO            479.530174
Diff_OPEN_DATE_CREDIT_SCORE       4001.383535
Diff_OPEN_DATE_BANKRUPTCY         4098.384847
```

In [ ]:

```
#Try 8: Removing Highest VIF Diff_OPEN_DATE_BANKRUPTCY
independent_variables9 = independent_variables8.drop(columns=['Diff_OPEN_DATE_BANKRUPT

result = vif(independent_variables9)
print(result)
```

```
                                          vif
LOAN_NBR                           437.282369
Charge-Off                         220.109481
NCO                                 26.784599
BALANCE                            185.566851
LIMIT                                3.085406
PRODUCT_NAME                         1.063173
TIMES_DELQ_30_DAYS                   7.845442
TIMES_DELQ_60_DAYS                  19.940571
TIMES_DELQ_90_DAYS                   8.759903
CREDIT_SCORE                        87.620646
MONTHS_OLD                        1044.760004
SEASONED_OVER_2YR                    4.115703
BANKPRUPTCY_FLAG                     1.065411
LOYALTY_LEVEL_ID                    16.822551
BANKRUPTCY_INDEX                     9.931938
HAS_DIRECT_DEP                       3.200760
CALC_CURRENT_MONTH_END_SHARE_BALANCE 3.360125
NEG_SHARE_BALANCE                    1.172997
LOYALTY_DUMMY_MEMBER                14.435365
CALC_PRIOR_MONTH_END_SHARE_BALANCE   3.399994
INT_ACCRUED                          2.407653
LGD%                               300.687255
new_england                          1.902461
Diff_OPEN_DATE_FIRST_CO            479.376367
Diff_OPEN_DATE_CREDIT_SCORE        513.559458
```

In [ ]:
```
# Try 9: Removing Highest VIF MONTHS_OLD

independent_variables10 = independent_variables9.drop(columns=['MONTHS_OLD'])

result = vif(independent_variables10)
print(result)
```

```
                                          vif
LOAN_NBR                           431.060115
Charge-Off                         219.846615
NCO                                 26.776366
BALANCE                            185.409043
LIMIT                                3.085216
PRODUCT_NAME                         1.063124
TIMES_DELQ_30_DAYS                   7.845049
TIMES_DELQ_60_DAYS                  19.938494
TIMES_DELQ_90_DAYS                   8.759832
CREDIT_SCORE                        87.376156
SEASONED_OVER_2YR                    4.115046
BANKPRUPTCY_FLAG                     1.065404
LOYALTY_LEVEL_ID                    16.819329
BANKRUPTCY_INDEX                     9.909713
HAS_DIRECT_DEP                       3.200501
CALC_CURRENT_MONTH_END_SHARE_BALANCE  3.360040
NEG_SHARE_BALANCE                    1.172972
LOYALTY_DUMMY_MEMBER                14.435160
CALC_PRIOR_MONTH_END_SHARE_BALANCE   3.399973
INT_ACCRUED                          2.407651
LGD%                               299.760816
new_england                          1.901708
Diff_OPEN_DATE_FIRST_CO            242.144834
Diff_OPEN_DATE_CREDIT_SCORE        200.892278
```

In [ ]:
```
# Try 10: Removing Highest VIF LOAN_NBR
independent_variables11 = independent_variables10.drop(columns=['LOAN_NBR'])

result = vif(independent_variables11)
print(result)
```

```
                                          vif
Charge-Off                          165.283888
NCO                                  26.776358
BALANCE                             131.911321
LIMIT                                 3.062543
PRODUCT_NAME                          1.062135
TIMES_DELQ_30_DAYS                    7.843635
TIMES_DELQ_60_DAYS                   19.937461
TIMES_DELQ_90_DAYS                    8.759524
CREDIT_SCORE                         75.521553
SEASONED_OVER_2YR                     4.081810
BANKPRUPTCY_FLAG                      1.065195
LOYALTY_LEVEL_ID                     16.058547
BANKRUPTCY_INDEX                      9.852302
HAS_DIRECT_DEP                        3.199567
CALC_CURRENT_MONTH_END_SHARE_BALANCE  3.359211
NEG_SHARE_BALANCE                     1.172367
LOYALTY_DUMMY_MEMBER                 13.691205
CALC_PRIOR_MONTH_END_SHARE_BALANCE    3.398705
INT_ACCRUED                           2.394872
LGD%                                 75.939949
new_england                           1.901248
Diff_OPEN_DATE_FIRST_CO             216.580168
Diff_OPEN_DATE_CREDIT_SCORE         180.025519
```

In [ ]:

```
# Try 11: Removing Highest VIF Diff_OPEN_DATE_FIRST_CO
independent_variables12 = independent_variables11.drop(columns=['Diff_OPEN_DATE_FIRST_

result = vif(independent_variables12)
print(result)
```

```
                                          vif
Charge-Off                          162.533170
NCO                                  26.744315
BALANCE                             128.474603
LIMIT                                 3.045384
PRODUCT_NAME                          1.061850
TIMES_DELQ_30_DAYS                    7.838154
TIMES_DELQ_60_DAYS                   19.935534
TIMES_DELQ_90_DAYS                    8.753000
CREDIT_SCORE                         73.811554
SEASONED_OVER_2YR                     4.075292
BANKPRUPTCY_FLAG                      1.064775
LOYALTY_LEVEL_ID                     16.029347
BANKRUPTCY_INDEX                      9.845185
HAS_DIRECT_DEP                        3.199126
CALC_CURRENT_MONTH_END_SHARE_BALANCE  3.359194
NEG_SHARE_BALANCE                     1.172180
LOYALTY_DUMMY_MEMBER                 13.666121
CALC_PRIOR_MONTH_END_SHARE_BALANCE    3.397766
INT_ACCRUED                           2.383650
LGD%                                 75.530455
new_england                           1.900900
Diff_OPEN_DATE_CREDIT_SCORE           3.081114
```

In [ ]:

```
# Try 12: Removing Highest VIF LGD%
independent_variables13 = independent_variables12.drop(columns=['LGD%'])

result = vif(independent_variables13)
print(result)
```

```
                                          vif
Charge-Off                          139.493117
NCO                                  26.734846
BALANCE                             106.283946
LIMIT                                 2.962241
PRODUCT_NAME                          1.060388
TIMES_DELQ_30_DAYS                    7.822922
TIMES_DELQ_60_DAYS                   19.921979
TIMES_DELQ_90_DAYS                    8.743214
CREDIT_SCORE                         40.804447
SEASONED_OVER_2YR                     4.013836
BANKPRUPTCY_FLAG                      1.063718
LOYALTY_LEVEL_ID                     13.800623
BANKRUPTCY_INDEX                      9.665325
HAS_DIRECT_DEP                        3.197886
CALC_CURRENT_MONTH_END_SHARE_BALANCE  3.358956
NEG_SHARE_BALANCE                     1.169178
LOYALTY_DUMMY_MEMBER                 11.748172
CALC_PRIOR_MONTH_END_SHARE_BALANCE    3.394500
INT_ACCRUED                           2.338159
new_england                           1.900600
Diff_OPEN_DATE_CREDIT_SCORE           3.080466
```

In [ ]:

```
# Try 13: Removing Highest VIF Charge-Off
independent_variables14 = independent_variables13.drop(columns=['Charge-Off'])

result = vif(independent_variables14)
print(result)
```

```
                                        vif
NCO                                19.637613
BALANCE                            23.656851
LIMIT                               2.961674
PRODUCT_NAME                        1.059796
TIMES_DELQ_30_DAYS                  7.822312
TIMES_DELQ_60_DAYS                 19.921451
TIMES_DELQ_90_DAYS                  8.743201
CREDIT_SCORE                       40.779020
SEASONED_OVER_2YR                   4.013822
BANKPRUPTCY_FLAG                    1.063717
LOYALTY_LEVEL_ID                   13.794260
BANKRUPTCY_INDEX                    9.641646
HAS_DIRECT_DEP                      3.197381
CALC_CURRENT_MONTH_END_SHARE_BALANCE  3.358408
NEG_SHARE_BALANCE                   1.169178
LOYALTY_DUMMY_MEMBER               11.746404
CALC_PRIOR_MONTH_END_SHARE_BALANCE  3.394336
INT_ACCRUED                         2.337383
new_england                         1.900598
Diff_OPEN_DATE_CREDIT_SCORE         3.080115
```

In [ ]:

```
# Try 14: Removing Highest VIF NCO
independent_variables15 = independent_variables14.drop(columns=['NCO'])

result = vif(independent_variables15)
print(result)
```

```
                                          vif
BALANCE                                4.772482
LIMIT                                  2.961512
PRODUCT_NAME                           1.059004
TIMES_DELQ_30_DAYS                     7.819853
TIMES_DELQ_60_DAYS                    19.921189
TIMES_DELQ_90_DAYS                     8.743201
CREDIT_SCORE                          40.766317
SEASONED_OVER_2YR                      4.013554
BANKPRUPTCY_FLAG                       1.063629
LOYALTY_LEVEL_ID                      13.794253
BANKRUPTCY_INDEX                       9.639849
HAS_DIRECT_DEP                         3.197374
CALC_CURRENT_MONTH_END_SHARE_BALANCE   3.358126
NEG_SHARE_BALANCE                      1.168939
LOYALTY_DUMMY_MEMBER                  11.745563
CALC_PRIOR_MONTH_END_SHARE_BALANCE     3.393851
INT_ACCRUED                            2.337241
new_england                            1.900598
Diff_OPEN_DATE_CREDIT_SCORE            3.080073
```

In [ ]:

```
# Try 15: Removing Highest VIF TIMES_DELQ_60_DAYS
independent_variables16 = independent_variables15.drop(columns=['TIMES_DELQ_60_DAYS'])

result = vif(independent_variables16)
print(result)
```

```
                                          vif
BALANCE                                4.761503
LIMIT                                  2.959343
PRODUCT_NAME                           1.058997
TIMES_DELQ_30_DAYS                     2.962347
TIMES_DELQ_90_DAYS                     2.090042
CREDIT_SCORE                          40.760115
SEASONED_OVER_2YR                      4.001950
BANKPRUPTCY_FLAG                       1.063576
LOYALTY_LEVEL_ID                      13.784848
BANKRUPTCY_INDEX                       9.639144
HAS_DIRECT_DEP                         3.197195
CALC_CURRENT_MONTH_END_SHARE_BALANCE   3.357956
NEG_SHARE_BALANCE                      1.168664
LOYALTY_DUMMY_MEMBER                  11.741618
CALC_PRIOR_MONTH_END_SHARE_BALANCE     3.393247
INT_ACCRUED                            2.336443
new_england                            1.900496
Diff_OPEN_DATE_CREDIT_SCORE            3.070774
```

In [ ]:

```
# Try 16: Removing Highest VIF LOYALTY_LEVEL_ID
independent_variables17 = independent_variables16.drop(columns=['LOYALTY_LEVEL_ID'])

result = vif(independent_variables17)
print(result)
```

```
                                          vif
BALANCE                              4.755328
LIMIT                                2.947350
PRODUCT_NAME                         1.058726
TIMES_DELQ_30_DAYS                   2.946079
TIMES_DELQ_90_DAYS                   2.088132
CREDIT_SCORE                        30.440933
SEASONED_OVER_2YR                    3.962334
BANKPRUPTCY_FLAG                     1.063388
BANKRUPTCY_INDEX                     9.565496
HAS_DIRECT_DEP                       2.521747
CALC_CURRENT_MONTH_END_SHARE_BALANCE 3.357614
NEG_SHARE_BALANCE                    1.158644
LOYALTY_DUMMY_MEMBER                 3.325672
CALC_PRIOR_MONTH_END_SHARE_BALANCE   3.393036
INT_ACCRUED                          2.327939
new_england                          1.880632
Diff_OPEN_DATE_CREDIT_SCORE          3.068528
```

Final Variables after applying VIF

After a careful assessment of multicollinearity using VIF (Variance Inflation Factor), we have identified and retained the most influential variables. The final set of variables includes those with VIF values below the threshold, ensuring a more robust and reliable regression analysis.

In [ ]:
```
# Try 17: Removing Highest VIF BANKRUPTCY_INDEX
independent_variables18 = independent_variables17.drop(columns=['BANKRUPTCY_INDEX'])

result = vif(independent_variables18)
print(result)
```

```
                                          vif
BALANCE                              4.471472
LIMIT                                2.946687
PRODUCT_NAME                         1.058319
TIMES_DELQ_30_DAYS                   2.931247
TIMES_DELQ_90_DAYS                   2.088071
CREDIT_SCORE                        16.399239
SEASONED_OVER_2YR                    3.643875
BANKPRUPTCY_FLAG                     1.056910
HAS_DIRECT_DEP                       2.502164
CALC_CURRENT_MONTH_END_SHARE_BALANCE 3.357466
NEG_SHARE_BALANCE                    1.158613
LOYALTY_DUMMY_MEMBER                 3.294407
CALC_PRIOR_MONTH_END_SHARE_BALANCE   3.392636
INT_ACCRUED                          2.324841
new_england                          1.811908
Diff_OPEN_DATE_CREDIT_SCORE          3.041646
```

MODEL 1 : LOGISTIC REGRESSION

Logistic regression is a **statistical model** specifically designed for binary classification tasks, where the dependent variable has only two possible outcomes. In our model, it serves the purpose of predicting whether defaulted members are likely to repay their debt to DCU or not.

The model assesses the likelihood of recovery being greater than 0, providing valuable insights into the repayment behavior of members.

In [ ]:
```
df3 = df.copy()
```

In [ ]:
```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from patsy import dmatrix
from sklearn.utils import resample
from sklearn.metrics import confusion_matrix, accuracy_score

# Target Variable
target_column = 'Recovery'

# Columns
allvars = df[['BALANCE', 'LIMIT', 'PRODUCT_NAME', 'TIMES_DELQ_30_DAYS', 'TIMES_DELQ_90

# Create design matrix
X = dmatrix("BALANCE + LIMIT + PRODUCT_NAME + TIMES_DELQ_30_DAYS + TIMES_DELQ_90_DAYS

# Create binary target variable
y = (df['Recovery'] > 0).astype(int)

# Resample data
positive_indices = np.where(y == 1)[0]
negative_indices = np.where(y == 0)[0]
random_negative_indices = np.random.choice(negative_indices, size=3305, replace=True)
resampled_indices = np.concatenate([positive_indices, random_negative_indices])

# Use resampled indices to get the corresponding labels and features
X_resampled = X.iloc[resampled_indices]
y_resampled = y.iloc[resampled_indices]

# Fit logistic regression model on the resampled data
glm_resampled = sm.GLM(y_resampled, X_resampled, family=sm.families.Binomial())
results_resampled = glm_resampled.fit()

# Summarize results for resampled data
print(results_resampled.summary())

# Predict probabilities using the fitted model on the original data
probs = results_resampled.predict(X_resampled)

# Set a threshold (e.g., 0.5) to classify probabilities into binary predictions
threshold = 0.2
binary_predictions = (probs > threshold).astype(int)

# Create a confusion matrix
conf_matrix = confusion_matrix(y_resampled, binary_predictions)

# Print the confusion matrix
print("Confusion Matrix:")
print(conf_matrix)

# Calculate accuracy
accuracy = accuracy_score(y_resampled, binary_predictions)
print("Accuracy:", accuracy)
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:               Recovery   No. Observations:                 6610
Model:                           GLM   Df Residuals:                     6594
Model Family:                Binomial   Df Model:                           15
Link Function:                  Logit   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -4506.3
Date:                Sat, 06 Jan 2024   Deviance:                       9012.5
Time:                        17:26:13   Pearson chi2:                 6.63e+03
No. Iterations:                     5   Pseudo R-squ. (CS):            0.02256
Covariance Type:            nonrobust
==============================================================================================
===================
                                            coef    std err          z      P>|z|
[0.025      0.975]
----------------------------------------------------------------------------------------------
-------------------
BALANCE                                -3.03e-05   9.02e-06     -3.360      0.001    -
4.8e-05   -1.26e-05
LIMIT                                 -1.689e-05   8.16e-06     -2.070      0.038   -
3.29e-05   -8.97e-07
PRODUCT_NAME                             -0.0375      0.169     -0.222      0.824
-0.368       0.293
TIMES_DELQ_30_DAYS                        0.0236      0.009      2.559      0.011
0.006       0.042
TIMES_DELQ_90_DAYS                       -0.0006      0.022     -0.029      0.977
-0.045       0.043
CREDIT_SCORE                              0.0004      0.000      3.417      0.001
0.000       0.001
SEASONED_OVER_2YR                         0.1305      0.062      2.120      0.034
0.010       0.251
BANKPRUPTCY_FLAG                          0.2531      0.185      1.369      0.171
-0.109       0.616
HAS_DIRECT_DEP                           -0.2647      0.067     -3.934      0.000
-0.397      -0.133
CALC_CURRENT_MONTH_END_SHARE_BALANCE  2.614e-05   1.35e-05      1.929      0.054   -
4.18e-07    5.27e-05
NEG_SHARE_BALANCE                        -0.1883      0.095     -1.987      0.047
-0.374      -0.003
LOYALTY_DUMMY_MEMBER                     -0.0909      0.063     -1.439      0.150
-0.215       0.033
CALC_PRIOR_MONTH_END_SHARE_BALANCE   -2.761e-05   1.35e-05     -2.044      0.041   -
5.41e-05   -1.14e-06
INT_ACCRUED                               0.0004      0.000      1.317      0.188
-0.000       0.001
new_england                              -0.2448      0.056     -4.350      0.000
-0.355      -0.135
Diff_OPEN_DATE_CREDIT_SCORE           1.498e-05   1.75e-05      0.856      0.392   -
1.93e-05    4.93e-05
==============================================================================================
===================
Confusion Matrix:
[[   7 3298]
 [   5 3300]]
Accuracy: 0.500302571860817
```

Editing the columns in Train and test sets after finding out VIF

In [ ]:
```
columns_to_keep = ['BALANCE', 'LIMIT', 'PRODUCT_NAME', 'TIMES_DELQ_30_DAYS',
                   'TIMES_DELQ_90_DAYS', 'CREDIT_SCORE', 'SEASONED_OVER_2YR',
                   'BANKPRUPTCY_FLAG', 'HAS_DIRECT_DEP', 'CALC_CURRENT_MONTH_END_SHARE
                   'NEG_SHARE_BALANCE', 'LOYALTY_DUMMY_MEMBER', 'CALC_PRIOR_MONTH_END_
                   'INT_ACCRUED', 'new_england', 'Diff_OPEN_DATE_CREDIT_SCORE', 'Recov

# Keep only the specified columns in train_set
train_set = train_set.loc[:, columns_to_keep]

# Keep only the specified columns in test_set_1
test_set_1 = test_set_1.loc[:, columns_to_keep]

# Keep only the specified columns in test_set_2
test_set_2 = test_set_2.loc[:, columns_to_keep]

# Keep only the specified columns in test_set_3
test_set_3 = test_set_3.loc[:, columns_to_keep]

# Keep only the specified columns in test_set_4
test_set_4 = test_set_4.loc[:, columns_to_keep]
```

In [ ]:
```
import statsmodels.api as sm
from patsy import dmatrix

# Target Column
target_column = 'Recovery'

# Variables
selected_columns = ['BALANCE', 'LIMIT', 'PRODUCT_NAME', 'TIMES_DELQ_30_DAYS', 'TIMES_D
                    'CREDIT_SCORE', 'SEASONED_OVER_2YR', 'BANKPRUPTCY_FLAG', 'HAS_DIR
                    'CALC_CURRENT_MONTH_END_SHARE_BALANCE', 'NEG_SHARE_BALANCE',
                    'LOYALTY_DUMMY_MEMBER', 'CALC_PRIOR_MONTH_END_SHARE_BALANCE',
                    'INT_ACCRUED', 'new_england', 'Diff_OPEN_DATE_CREDIT_SCORE']

# Create design matrix for the selected columns in the training set
X_train = dmatrix("BALANCE + LIMIT + PRODUCT_NAME + TIMES_DELQ_30_DAYS + TIMES_DELQ_90

# Reset index of X_train to ensure proper alignment
X_train = X_train.reset_index(drop=True)

# Create binary target variable for the training set
y_train = (train_set['Recovery'] > 0).astype(int)

# Set the index of y_train to match the index of X_train
y_train.index = X_train.index

# Fit logistic regression model on the training data
glm_train = sm.GLM(y_train, X_train, family=sm.families.Binomial())
results_train = glm_train.fit()

# Summarize results for the training data
print(results_train.summary())
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:               Recovery   No. Observations:                 8730
Model:                            GLM   Df Residuals:                     8714
Model Family:                Binomial   Df Model:                           15
Link Function:                  Logit   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -3702.0
Date:                Sat, 06 Jan 2024   Deviance:                       7404.0
Time:                        17:26:13   Pearson chi2:                 8.79e+03
No. Iterations:                     6   Pseudo R-squ. (CS):            0.01887
Covariance Type:            nonrobust
==============================================================================================
                                             coef    std err          z      P>|z|
[0.025      0.975]
----------------------------------------------------------------------------------------------
BALANCE                               -1.544e-05   1.03e-05     -1.502      0.133   -
3.56e-05    4.71e-06
LIMIT                                 -2.234e-05   1.03e-05     -2.176      0.030   -
4.25e-05   -2.21e-06
PRODUCT_NAME                             -0.1684      0.270     -0.623      0.533
-0.698       0.361
TIMES_DELQ_30_DAYS                        0.0221      0.012      1.780      0.075
-0.002       0.046
TIMES_DELQ_90_DAYS                        0.0076      0.035      0.217      0.828
-0.061       0.076
CREDIT_SCORE                             -0.0023      0.000    -20.606      0.000
-0.003      -0.002
SEASONED_OVER_2YR                        -0.1364      0.075     -1.809      0.070
-0.284       0.011
BANKPRUPTCY_FLAG                          0.5974      0.178      3.358      0.001
0.249       0.946
HAS_DIRECT_DEP                           -0.3714      0.074     -5.043      0.000
-0.516      -0.227
CALC_CURRENT_MONTH_END_SHARE_BALANCE  1.798e-05   2.32e-05      0.776      0.438   -
2.74e-05    6.34e-05
NEG_SHARE_BALANCE                        -0.3767      0.129     -2.921      0.003
-0.629      -0.124
LOYALTY_DUMMY_MEMBER                     -0.0365      0.068     -0.539      0.590
-0.169       0.096
CALC_PRIOR_MONTH_END_SHARE_BALANCE   -2.315e-05   2.04e-05     -1.135      0.256   -
6.31e-05    1.68e-05
INT_ACCRUED                           -4.164e-05      0.000     -0.115      0.908
-0.001       0.001
new_england                              -0.3681      0.068     -5.381      0.000
-0.502      -0.234
Diff_OPEN_DATE_CREDIT_SCORE           5.521e-05    2.1e-05      2.630      0.009
1.41e-05    9.64e-05
==============================================================================================
```

In [ ]:

```python
from sklearn.metrics import confusion_matrix
confusion_matrix(y_train, train_set['BANKPRUPTCY_FLAG'])
```

Out[ ]:

```
array([[7212,  153],
       [1319,   46]], dtype=int64)
```

In [ ]:

```python
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
# Predict probabilities using the fitted model on the original data
probs_train = results_train.predict(exog=X_train)

# Set a threshold (e.g., 0.2) to classify probabilities into binary predictions for th
threshold = 0.2
binary_predictions_train = (probs_train > threshold).astype(int)

y_train = (train_set['Recovery'] > 0).astype(int)

# Reset indices to ensure they match
y_train.index = binary_predictions_train.index

# Create a confusion matrix for the training set
conf_matrix_train = confusion_matrix(y_train, binary_predictions_train)

# Print the confusion matrix for the training set
print("Confusion Matrix (Training Set):")
print(conf_matrix_train)
```

```
Confusion Matrix (Training Set):
[[5832 1533]
 [ 909  456]]
```

In [ ]:

```
# Calculate accuracy
accuracy = accuracy_score(y_train, binary_predictions_train)

# Calculate sensitivity
sensitivity = conf_matrix_train[1, 1] / (conf_matrix_train[1, 1] + conf_matrix_train[1

# Calculate specificity
specificity = conf_matrix_train[0, 0] / (conf_matrix_train[0, 0] + conf_matrix_train[0

# Calculate prevalence
prevalence = (conf_matrix_train[1, 1] + conf_matrix_train[1, 0]) / np.sum(conf_matrix_

# Print
print("Accuracy:", accuracy)
print("Sensitivity (True Positive Rate):", sensitivity)
print("Specificity (True Negative Rate):", specificity)
print("Prevalence:", prevalence)
```

```
Accuracy: 0.7202749140893471
Sensitivity (True Positive Rate): 0.33406593406593404
Specificity (True Negative Rate): 0.7918533604887984
Prevalence: 0.1563573883161512
```

In [ ]:

```
selected_features = ['BALANCE', 'LIMIT', 'PRODUCT_NAME', 'TIMES_DELQ_30_DAYS', 'TIMES_
                     'CREDIT_SCORE', 'SEASONED_OVER_2YR', 'BANKPRUPTCY_FLAG', 'HAS_DIF
                     'CALC_CURRENT_MONTH_END_SHARE_BALANCE', 'NEG_SHARE_BALANCE',
                     'LOYALTY_DUMMY_MEMBER', 'CALC_PRIOR_MONTH_END_SHARE_BALANCE',
                     'INT_ACCRUED', 'new_england', 'Diff_OPEN_DATE_CREDIT_SCORE']

# Create design matrix for the selected features in the test set
X_test = dmatrix("BALANCE + LIMIT + PRODUCT_NAME + TIMES_DELQ_30_DAYS + TIMES_DELQ_90_

# Reset index of X_test to ensure proper alignment
X_test = X_test.reset_index(drop=True)

# Predict probabilities using the fitted model on the test set
probs_test = results_train.predict(exog=X_test)
```

```
# Set the same threshold used for training to classify probabilities into binary predi
binary_predictions_test = (probs_test > threshold).astype(int)

y_test = (test_set_1['Recovery'] > 0).astype(int)

# Reset indices to ensure they match
y_test.index = binary_predictions_test.index

# Create a confusion matrix for the test set
conf_matrix_test = confusion_matrix(y_test, binary_predictions_test)

# Print the confusion matrix for the test set
print("Confusion Matrix (Test Set 1):")
print(conf_matrix_test)
```

```
Confusion Matrix (Test Set 1):
[[5460 1433]
 [1006  414]]
```

In [ ]:

```
# Calculate sensitivity, specificity, prevalence, and accuracy for the test set

# Accuracy
accuracy_test = accuracy_score(y_test, binary_predictions_test)

# Sensitivity (True Positive Rate or Recall)
sensitivity_test = conf_matrix_test[1, 1] / (conf_matrix_test[1, 1] + conf_matrix_test

# Specificity (True Negative Rate)
specificity_test = conf_matrix_test[0, 0] / (conf_matrix_test[0, 0] + conf_matrix_test

# Prevalence
prevalence_test = (conf_matrix_test[1, 1] + conf_matrix_test[1, 0]) / np.sum(conf_matr


# Print the results for the test set
print("Accuracy:", accuracy_test)
print("Sensitivity (True Positive Rate):", sensitivity_test)
print("Specificity (True Negative Rate):", specificity_test)
print("Prevalence:", prevalence_test)
```

```
Accuracy: 0.7066041140382533
Sensitivity (True Positive Rate): 0.2915492957746479
Specificity (True Negative Rate): 0.7921079355868272
Prevalence: 0.17081679297485866
```

In [ ]:

```
selected_features = ['BALANCE', 'LIMIT', 'PRODUCT_NAME', 'TIMES_DELQ_30_DAYS', 'TIMES_
                     'CREDIT_SCORE', 'SEASONED_OVER_2YR', 'BANKPRUPTCY_FLAG', 'HAS_DIF
                     'CALC_CURRENT_MONTH_END_SHARE_BALANCE', 'NEG_SHARE_BALANCE',
                     'LOYALTY_DUMMY_MEMBER', 'CALC_PRIOR_MONTH_END_SHARE_BALANCE',
                     'INT_ACCRUED', 'new_england', 'Diff_OPEN_DATE_CREDIT_SCORE']

# Create design matrix for the selected features in the test set
X_test_2 = dmatrix("BALANCE + LIMIT + PRODUCT_NAME + TIMES_DELQ_30_DAYS + TIMES_DELQ_9

# Reset index of X_test_2 to ensure proper alignment
X_test_2 = X_test_2.reset_index(drop=True)

# Predict probabilities using the fitted model on the test set
probs_test_2 = results_train.predict(exog=X_test_2)

# Set the same threshold used for training to classify probabilities into binary predi
```

```
binary_predictions_test_2 = (probs_test_2 > threshold).astype(int)


y_test_2 = (test_set_2['Recovery'] > 0).astype(int)

# Reset indices to ensure they match
y_test_2.index = binary_predictions_test_2.index

# Create a confusion matrix for the test set
conf_matrix_test_2 = confusion_matrix(y_test_2, binary_predictions_test_2)

# Print the confusion matrix for the test set
print("Confusion Matrix (Test Set 2):")
print(conf_matrix_test_2)
```

```
Confusion Matrix (Test Set 2):
[[1064  278]
 [ 114   48]]
```

In [ ]:

```
# Calculate sensitivity, specificity, prevalence, and accuracy for the second test set

# Accuracy
accuracy_test_2 = accuracy_score(y_test_2, binary_predictions_test_2)

# Sensitivity (True Positive Rate or Recall)
sensitivity_test_2 = conf_matrix_test_2[1, 1] / (conf_matrix_test_2[1, 1] + conf_matri

# Specificity (True Negative Rate)
specificity_test_2 = conf_matrix_test_2[0, 0] / (conf_matrix_test_2[0, 0] + conf_matri

# Prevalence
prevalence_test_2 = (conf_matrix_test_2[1, 1] + conf_matrix_test_2[1, 0]) / np.sum(cor

# Print the results for the second test set
print("Accuracy:", accuracy_test_2)
print("Sensitivity (True Positive Rate):", sensitivity_test_2)
print("Specificity (True Negative Rate):", specificity_test_2)
print("Prevalence:", prevalence_test_2)
```

```
Accuracy: 0.7393617021276596
Sensitivity (True Positive Rate): 0.2962962962962963
Specificity (True Negative Rate): 0.7928464977645305
Prevalence: 0.1077127659574468
```

In [ ]:

```
selected_features = ['BALANCE', 'LIMIT', 'PRODUCT_NAME', 'TIMES_DELQ_30_DAYS', 'TIMES_
                     'CREDIT_SCORE', 'SEASONED_OVER_2YR', 'BANKPRUPTCY_FLAG', 'HAS_DIF
                     'CALC_CURRENT_MONTH_END_SHARE_BALANCE', 'NEG_SHARE_BALANCE',
                     'LOYALTY_DUMMY_MEMBER', 'CALC_PRIOR_MONTH_END_SHARE_BALANCE',
                     'INT_ACCRUED', 'new_england', 'Diff_OPEN_DATE_CREDIT_SCORE']

# Create design matrix for the selected features in the test set
X_test_3 = dmatrix("BALANCE + LIMIT + PRODUCT_NAME + TIMES_DELQ_30_DAYS + TIMES_DELQ_9

# Reset index of X_test_3 to ensure proper alignment
X_test_3 = X_test_3.reset_index(drop=True)

# Predict probabilities using the fitted model on the test set
probs_test_3 = results_train.predict(exog=X_test_3)

# Set the same threshold used for training to classify probabilities into binary predi
binary_predictions_test_3 = (probs_test_3 > threshold).astype(int)
```

```
y_test_3 = (test_set_3['Recovery'] > 0).astype(int)

# Reset indices to ensure they match
y_test_3.index = binary_predictions_test_3.index

# Create a confusion matrix for the test set
conf_matrix_test_3 = confusion_matrix(y_test_3, binary_predictions_test_3)

# Print the confusion matrix for the test set
print("Confusion Matrix (Test Set 3):")
print(conf_matrix_test_3)

# Calculate accuracy for the test set
accuracy_test_3 = accuracy_score(y_test_3, binary_predictions_test_3)
print("Accuracy (Test Set 3):", accuracy_test_3)
```

```
Confusion Matrix (Test Set 3):
[[1666  595]
 [  94   61]]
Accuracy (Test Set 3): 0.714817880794702
```

In [ ]:

```
# Calculate sensitivity, specificity, prevalence, and accuracy for the second test set

# Accuracy
accuracy_test_3 = accuracy_score(y_test_3, binary_predictions_test_3)

# Sensitivity (True Positive Rate or Recall)
sensitivity_test_3 = conf_matrix_test_3[1, 1] / (conf_matrix_test_3[1, 1] + conf_matri

# Specificity (True Negative Rate)
specificity_test_3 = conf_matrix_test_3[0, 0] / (conf_matrix_test_3[0, 0] + conf_matri

# Prevalence
prevalence_test_3 = (conf_matrix_test_3[1, 1] + conf_matrix_test_3[1, 0]) / np.sum(cor

# Print the results for the second test set
print("Accuracy:", accuracy_test_3)
print("Sensitivity (True Positive Rate):", sensitivity_test_3)
print("Specificity (True Negative Rate):", specificity_test_3)
print("Prevalence:", prevalence_test_3)
```

```
Accuracy: 0.714817880794702
Sensitivity (True Positive Rate): 0.3935483870967742
Specificity (True Negative Rate): 0.7368421052631579
Prevalence: 0.06415562913907284
```

In [ ]:

```
selected_features = ['BALANCE', 'LIMIT', 'PRODUCT_NAME', 'TIMES_DELQ_30_DAYS', 'TIMES_
                     'CREDIT_SCORE', 'SEASONED_OVER_2YR', 'BANKPRUPTCY_FLAG', 'HAS_DIF
                     'CALC_CURRENT_MONTH_END_SHARE_BALANCE', 'NEG_SHARE_BALANCE',
                     'LOYALTY_DUMMY_MEMBER', 'CALC_PRIOR_MONTH_END_SHARE_BALANCE',
                     'INT_ACCRUED', 'new_england', 'Diff_OPEN_DATE_CREDIT_SCORE']

# Create design matrix for the selected features in the test set
X_test_4 = dmatrix("BALANCE + LIMIT + PRODUCT_NAME + TIMES_DELQ_30_DAYS + TIMES_DELQ_9

# Reset index of X_test_4 to ensure proper alignment
X_test_4 = X_test_4.reset_index(drop=True)

# Predict probabilities using the fitted model on the test set
```

```
probs_test_4 = results_train.predict(exog=X_test_4)

# Set the same threshold used for training to classify probabilities into binary predi
binary_predictions_test_4 = (probs_test_4 > threshold).astype(int)

y_test_4 = (test_set_4['Recovery'] > 0).astype(int)

# Reset indices to ensure they match
y_test_4.index = binary_predictions_test_4.index

# Create a confusion matrix for the test set
conf_matrix_test_4 = confusion_matrix(y_test_4, binary_predictions_test_4)

# Print the confusion matrix for the test set
print("Confusion Matrix (Test Set 4):")
print(conf_matrix_test_4)
```

```
Confusion Matrix (Test Set 4):
[[476 153]
 [ 25  13]]
```

In [ ]:

```
# Calculate sensitivity, specificity, prevalence, and accuracy for the second test set

# Accuracy
accuracy_test_4 = accuracy_score(y_test_4, binary_predictions_test_4)

# Sensitivity (True Positive Rate or Recall)
sensitivity_test_4 = conf_matrix_test_4[1, 1] / (conf_matrix_test_4[1, 1] + conf_matri

# Specificity (True Negative Rate)
specificity_test_4 = conf_matrix_test_4[0, 0] / (conf_matrix_test_4[0, 0] + conf_matri

# Prevalence
prevalence_test_4 = (conf_matrix_test_4[1, 1] + conf_matrix_test_4[1, 0]) / np.sum(cor

# Print the results for the second test set
print("Accuracy:", accuracy_test_4)
print("Sensitivity (True Positive Rate):", sensitivity_test_4)
print("Specificity (True Negative Rate):", specificity_test_4)
print("Prevalence:", prevalence_test_4)
```

```
Accuracy: 0.7331334332833583
Sensitivity (True Positive Rate): 0.34210526315789475
Specificity (True Negative Rate): 0.7567567567567568
Prevalence: 0.05697151424287856
```

MODEL 2 : LINEAR REGRESSION

In linear regression, we model the recovery amount as the dependent variable. We consider one or more independent variables believed to influence the recovery amount. The model assumes a linear relationship between these variables and the recovery amount.

Key Components:

1. **Dependent Variable:**

    - The recovery amount is designated as the dependent variable.

2. **Independent Variables:**
   - One or more independent variables are chosen based on the belief that they impact the recovery amount.
3. **Assumption of Linearity:**
   - Linear regression assumes a linear relationship between the chosen independent variables and the recovery amount.
4. **Residuals:**
   - The model assumes that the residuals, representing differences between actual and predicted values, follow a normal distribution.

Objective:

The goal is to understand and quantify how changes in the independent variables are associated with changes in the recovery amount, providing insights for prediction and analysis.

In [ ]:
```
# Using only the observation where Recovery is > $0.

df1 = df.copy()
df1 = df[df['Recovery'] > 0]
print(df1)

# List of columns to keep
columns_to_keep = [
    'LIMIT',
    'PRODUCT_NAME',
    'TIMES_DELQ_30_DAYS',
    'TIMES_DELQ_90_DAYS',
    'CREDIT_SCORE',
    'SEASONED_OVER_2YR',
    'BANKPRUPTCY_FLAG',
    'HAS_DIRECT_DEP',
    'CALC_CURRENT_MONTH_END_SHARE_BALANCE',
    'NEG_SHARE_BALANCE',
    'LOYALTY_DUMMY_MEMBER',
    'CALC_PRIOR_MONTH_END_SHARE_BALANCE',
    'INT_ACCRUED',
    'new_england',
    'Diff_OPEN_DATE_CREDIT_SCORE',
    'BALANCE',
    'Recovery'
]
```

```
       LoanIDNumber  LOAN_NBR  Charge-Off  Recovery       NCO MONTH_END_DATE  \
14            39069       142      999.87    999.87      0.00     2014-08-31
17            39072       141     2462.07   2462.07      0.00     2013-04-30
27            39082       142    25266.88   6033.25  19233.63     2017-04-30
29            39084       141     2922.61   2767.77    154.84     2014-06-30
47            39102       141      932.62     25.44    907.18     2016-02-29
...             ...       ...         ...       ...       ...            ...
29753         10294       141        0.00    100.00   -100.00     2012-11-30
29766         10304       141        0.00   3320.30  -3320.30     2013-10-31
29769         10307       141        0.00    505.49   -505.49     2012-10-31
29789         10327       141      111.44    111.44      0.00     2020-01-31
29795         10333       141    12997.20   6093.75   6903.45     2014-03-31

        BALANCE    LIMIT  LOAN_TYPE  PRODUCT_NAME  ...  new_england  \
14       999.87   1000.0         66             0  ...            0
17      2462.07      0.0         66             0  ...            0
27     25266.88      0.0         66             0  ...            0
29      2922.61      0.0         66             0  ...            1
47       932.62      0.0         66             0  ...            0
...         ...      ...        ...           ...  ...          ...
29753    964.02      0.0         66             0  ...            1
29766   3327.56      0.0         66             0  ...            1
29769   4998.32   5000.0         67             1  ...            0
29789    111.44  18500.0         66             0  ...            1
29795  12997.20  13000.0         66             0  ...            1

       Diff_FIRST_CO_MONTH_END  Diff_FIRST_CO_CREDIT_SCORE  \
14                         212                         274
17                         214                         333
27                         488                         607
29                         274                         274
47                         275                         334
...                        ...                         ...
29753                      577                        2373
29766                      212                         264
29769                      395                         518
29789                      243                         274
29795                      334                         423

       Diff_FIRST_CO_BANKRUPTCY  Diff_MONTH_END_CREDIT_SCORE  \
14                          274                           62
17                          333                          119
27                          607                          119
29                          274                            0
47                          334                           59
...                         ...                          ...
29753                       730                         1796
29766                       264                           52
29769                       518                          123
29789                       274                           31
29795                       423                           89

       Diff_MONTH_END_BANKRUPTCY  Diff_OPEN_DATE_FIRST_CO  \
14                            62                     2121
17                           119                     5981
27                           119                     8270
29                             0                     4295
47                            59                     3991
...                          ...                      ...
29753                        153                     7661
```

```
29766                          52                      9182
29769                         123                      6826
29789                          31                      9451
29795                          89                      6905

       Diff_OPEN_DATE_MONTH_END  Diff_OPEN_DATE_CREDIT_SCORE  \
14                         1909                         1847
17                         5767                         5648
27                         7782                         7663
29                         4021                         4021
47                         3716                         3657
...                         ...                          ...
29753                      7084                         5288
29766                      8970                         8918
29769                      6431                         6308
29789                      9208                         9177
29795                      6571                         6482

       Diff_OPEN_DATE_BANKRUPTCY
14                          1847
17                          5648
27                          7663
29                          4021
47                          3657
...                          ...
29753                       6931
29766                       8918
29769                       6308
29789                       9177
29795                       6482

[3305 rows x 37 columns]
```

In [ ]:

```
# For Visualizations
fp = r"New_updated.xlsx"
df1.to_excel(fp, sheet_name='Edit 3')


print("Modified data saved to")
```

```
Modified data saved to
```

In [ ]:

```
# Create the new DataFrame with selected columns
df2 = df1[columns_to_keep]
print(df2)
```

```
         LIMIT  PRODUCT_NAME  TIMES_DELQ_30_DAYS  TIMES_DELQ_90_DAYS  \
14      1000.0            0                   0                   0
17         0.0            0                   1                   0
27         0.0            0                   6                   0
29         0.0            0                   1                   0
47         0.0            0                   5                   0
...        ...          ...                 ...                 ...
29753      0.0            0                   1                   1
29766      0.0            0                   5                   0
29769   5000.0            1                   0                   0
29789  18500.0            0                   0                   0
29795  13000.0            0                   0                   0

       CREDIT_SCORE  SEASONED_OVER_2YR  BANKPRUPTCY_FLAG  HAS_DIRECT_DEP  \
14             620                  1                 0               0
17             519                  1                 0               1
27             539                  1                 0               0
29             535                  1                 1               0
47             563                  1                 0               1
...            ...                ...               ...             ...
29753          650                  1                 1               1
29766          550                  1                 0               1
29769          699                  1                 0               1
29789          813                  1                 0               1
29795          580                  1                 0               0

       CALC_CURRENT_MONTH_END_SHARE_BALANCE  NEG_SHARE_BALANCE  \
14                                  -601.94                  1
17                                     5.83                  0
27                                   -62.80                  1
29                                    46.69                  0
47                                    12.27                  0
...                                     ...                ...
29753                                874.86                  0
29766                                 50.98                  0
29769                                1387.51                  0
29789                               27528.66                  0
29795                                   8.56                  0

       LOYALTY_DUMMY_MEMBER  CALC_PRIOR_MONTH_END_SHARE_BALANCE  INT_ACCRUED  \
14                        0                             1607.52        16.69
17                        0                              160.37       254.15
27                        0                               27.17       282.97
29                        0                               45.93        28.72
47                        0                               19.10       219.02
...                     ...                                 ...          ...
29753                     0                             1092.76        11.93
29766                     0                              109.98        55.22
29769                     0                              408.59        55.01
29789                     0                            26349.08         1.06
29795                     1                                8.54        89.06

       new_england  Diff_OPEN_DATE_CREDIT_SCORE   BALANCE  Recovery
14               0                         1847    999.87    999.87
17               0                         5648   2462.07   2462.07
27               0                         7663  25266.88   6033.25
29               1                         4021   2922.61   2767.77
47               0                         3657    932.62     25.44
...            ...                          ...       ...       ...
29753            1                         5288    964.02    100.00
```

```
29766            1                        8918   3327.56   3320.30
29769            0                        6308   4998.32    505.49
29789            1                        9177    111.44    111.44
29795            1                        6482  12997.20   6093.75

[3305 rows x 17 columns]
```

In [ ]:

```
# Result without training and testing
X = df2.drop('Recovery', axis=1)
y = df1['Recovery']
X = sm.add_constant(X)  # Add a constant term (intercept) to the independent variables
print(X.shape)
print(y.shape)

model = sm.OLS(y, X)
results = model.fit()
print(results.summary())
```

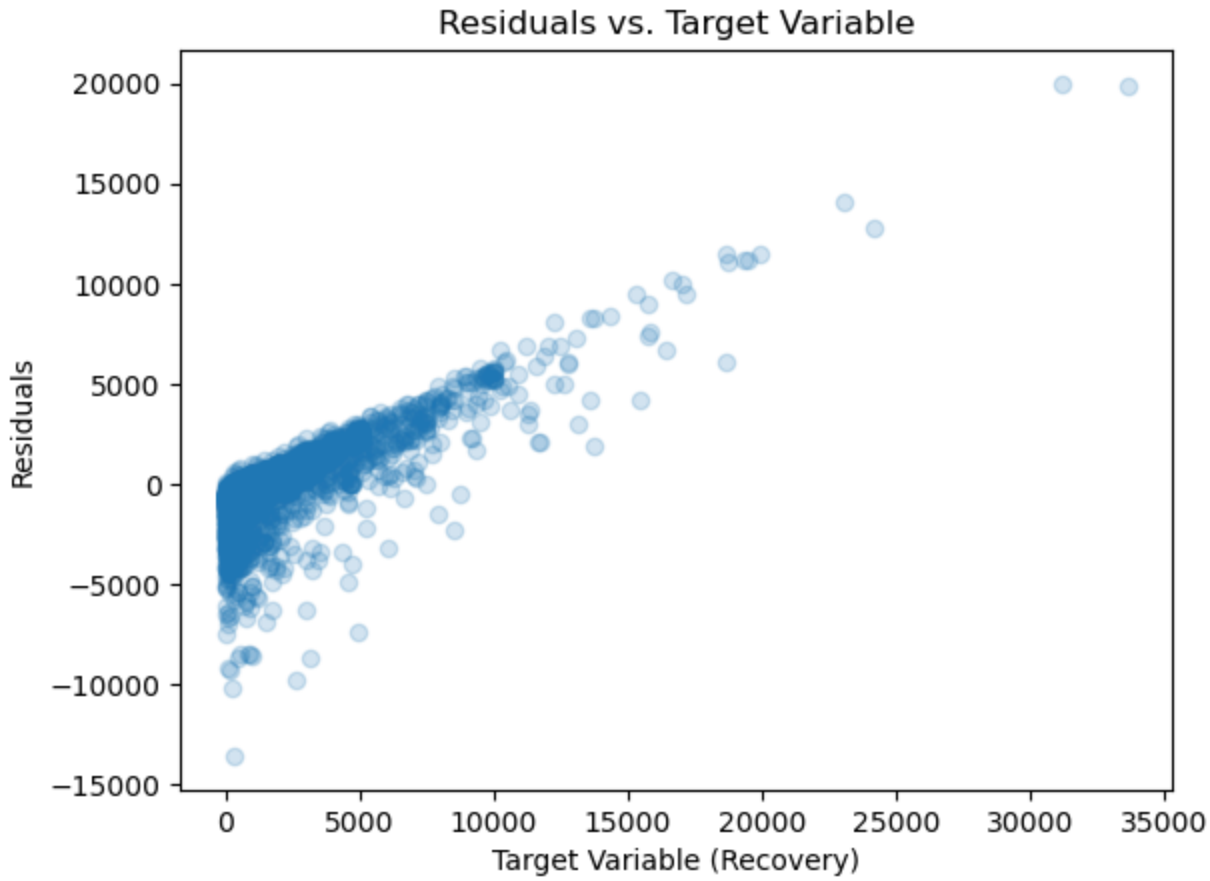
```
(3305, 17)
(3305,)
                            OLS Regression Results
==============================================================================
Dep. Variable:               Recovery   R-squared:                       0.412
Model:                            OLS   Adj. R-squared:                  0.409
Method:                 Least Squares   F-statistic:                     143.8
Date:                Sat, 06 Jan 2024   Prob (F-statistic):               0.00
Time:                        17:26:14   Log-Likelihood:                -29805.
No. Observations:                3305   AIC:                         5.964e+04
Df Residuals:                    3288   BIC:                         5.975e+04
Df Model:                          16
Covariance Type:            nonrobust
==================================================================================
===================
                                          coef    std err          t      P>|t|
[0.025      0.975]
----------------------------------------------------------------------------------
-------------------
const                                 -197.2726    303.514     -0.650      0.516    -
792.369     397.824
LIMIT                                    0.0340      0.012      2.736      0.006
0.010       0.058
PRODUCT_NAME                           773.2779    251.491      3.075      0.002
280.184    1266.372
TIMES_DELQ_30_DAYS                      29.9582     12.130      2.470      0.014
6.174      53.742
TIMES_DELQ_90_DAYS                     -57.2059     27.264     -2.098      0.036    -
110.662      -3.749
CREDIT_SCORE                             0.8706      0.503      1.731      0.083
-0.115       1.857
SEASONED_OVER_2YR                      124.6085     87.401      1.426      0.154
-46.758     295.975
BANKPRUPTCY_FLAG                      -501.7475    249.328     -2.012      0.044    -
990.601     -12.894
HAS_DIRECT_DEP                         -65.9054     95.819     -0.688      0.492    -
253.777     121.966
CALC_CURRENT_MONTH_END_SHARE_BALANCE     0.0369      0.020      1.813      0.070
-0.003       0.077
NEG_SHARE_BALANCE                      -66.2450    140.837     -0.470      0.638    -
342.382     209.892
LOYALTY_DUMMY_MEMBER                    -8.5465     88.108     -0.097      0.923    -
181.299     164.206
CALC_PRIOR_MONTH_END_SHARE_BALANCE      -0.0375      0.020     -1.872      0.061
-0.077       0.002
INT_ACCRUED                              0.2349      0.465      0.505      0.614
-0.678       1.147
new_england                            377.7040     81.257      4.648      0.000
218.384     537.024
Diff_OPEN_DATE_CREDIT_SCORE             -0.0618      0.026     -2.343      0.019
-0.114      -0.010
BALANCE                                  0.3625      0.013     28.764      0.000
0.338       0.387
==============================================================================
Omnibus:                     1017.771   Durbin-Watson:                   2.017
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            20642.980
Skew:                           0.957   Prob(JB):                         0.00
Kurtosis:                      15.093   Cond. No.                     6.94e+04
==============================================================================
```

```
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly spec
ified.
[2] The condition number is large, 6.94e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

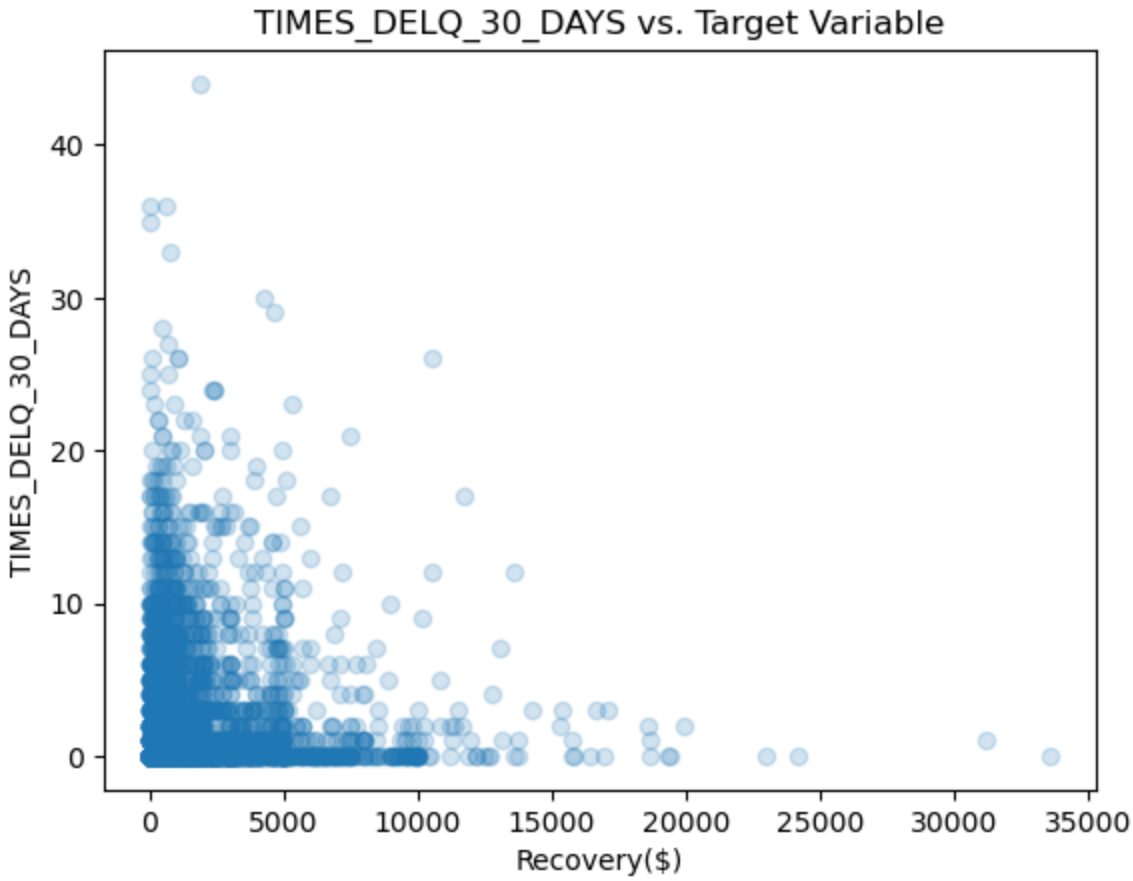
In [ ]:

```python
# Calculate the predicted values
predicted_Recovery = results.fittedvalues

# Add the predicted_Recovery column to df2
df2['predicted_Recovery'] = predicted_Recovery


# Calculate the residuals
residuals = y - predicted_Recovery

# Create a scatter plot of residuals vs. the target variable
import matplotlib.pyplot as plt

plt.scatter(y, residuals, alpha=0.2)
plt.xlabel('Target Variable (Recovery)')
plt.ylabel('Residuals')
plt.title('Residuals vs. Target Variable')
plt.show()
```

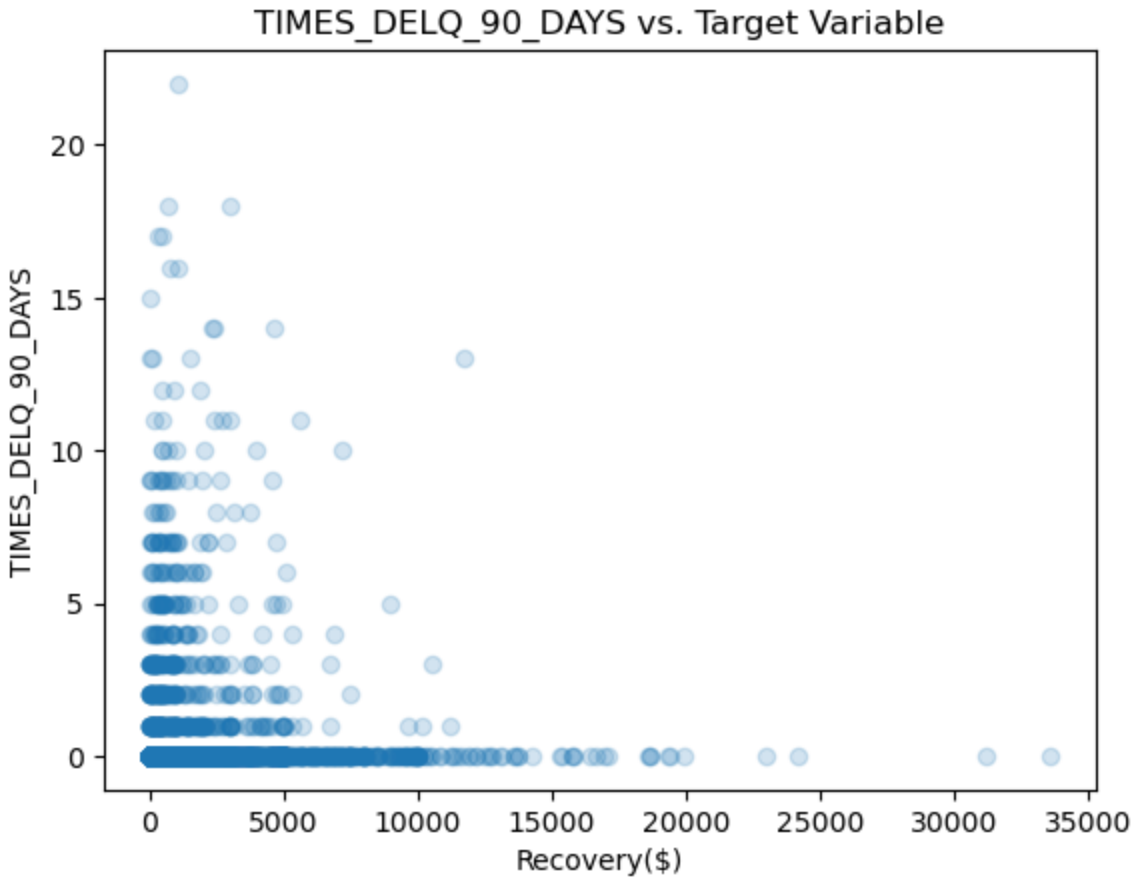
```
C:\Users\19783\AppData\Local\Temp\ipykernel_16300\1491640429.py:5: SettingWithCopyWar
ning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
er_guide/indexing.html#returning-a-view-versus-a-copy
  df2['predicted_Recovery'] = predicted_Recovery
```



In [ ]:
```
# scatter plot of residuals vs. the target variable
import matplotlib.pyplot as plt

plt.scatter(y, df2['TIMES_DELQ_30_DAYS'], alpha=0.2)
plt.xlabel('Recovery($)')
plt.ylabel('TIMES_DELQ_30_DAYS')
plt.title('TIMES_DELQ_30_DAYS vs. Target Variable')
plt.show()
```



In [ ]:
```
# scatter plot of residuals vs. the target variable
import matplotlib.pyplot as plt

plt.scatter(y, df2['TIMES_DELQ_90_DAYS'], alpha=0.2)
plt.xlabel('Recovery($)')
plt.ylabel('TIMES_DELQ_90_DAYS')
plt.title('TIMES_DELQ_90_DAYS vs. Target Variable')
plt.show()
```



In [ ]:
```
# Create new columns TrueLGD & PredictedLGD
df2['TrueLGD'] = 1 - (df2['Recovery'] / df2['BALANCE'])
df2['PredictedLGD'] = 1 - (df2['predicted_Recovery'] / df2['BALANCE'])
```

```
C:\Users\19783\AppData\Local\Temp\ipykernel_16300\3276009500.py:2: SettingWithCopyWar
ning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
er_guide/indexing.html#returning-a-view-versus-a-copy
  df2['TrueLGD'] = 1 - (df2['Recovery'] / df2['BALANCE'])
C:\Users\19783\AppData\Local\Temp\ipykernel_16300\3276009500.py:3: SettingWithCopyWar
ning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
er_guide/indexing.html#returning-a-view-versus-a-copy
  df2['PredictedLGD'] = 1 - (df2['predicted_Recovery'] / df2['BALANCE'])
```

Add the columns to the train & test

In [ ]:
```
print(df2[['TrueLGD', 'predicted_Recovery', 'PredictedLGD']])
```

```
        TrueLGD  predicted_Recovery  PredictedLGD
14     0.000000          604.601951      0.395319
17     0.000000          940.635639      0.617949
27     0.761219         9259.206318      0.633544
29     0.052980         1116.768197      0.617887
47     0.972722          664.662129      0.287317
...         ...                 ...           ...
29753  0.896268          292.786562      0.696286
29766  0.002182         1533.655649      0.539105
29769  0.898868         2884.423513      0.422921
29789  0.000000         1077.026585     -8.664632
29795  0.531149         5576.042251      0.570981

[3305 rows x 3 columns]
```

In [ ]:
```
df2
```

Out[ ]:

	LIMIT	PRODUCT_NAME	TIMES_DELQ_30_DAYS	TIMES_DELQ_90_DAYS	CREDIT_SCORE	SEASO
14	1000.0	0	0	0	620	
17	0.0	0	1	0	519	
27	0.0	0	6	0	539	
29	0.0	0	1	0	535	
47	0.0	0	5	0	563	
...	...	...	...	...	...	
29753	0.0	0	1	1	650	
29766	0.0	0	5	0	550	
29769	5000.0	1	0	0	699	
29789	18500.0	0	0	0	813	
29795	13000.0	0	0	0	580	

3305 rows × 20 columns

In [ ]:
```
med = df[df['Recovery'] > 0]
df2['MONTH_END_DATE'] = med['MONTH_END_DATE']
df2
```

```
C:\Users\19783\AppData\Local\Temp\ipykernel_16300\77591871.py:2: SettingWithCopyWarni
ng:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
er_guide/indexing.html#returning-a-view-versus-a-copy
  df2['MONTH_END_DATE'] = med['MONTH_END_DATE']
```

Out[ ]:

	LIMIT	PRODUCT_NAME	TIMES_DELQ_30_DAYS	TIMES_DELQ_90_DAYS	CREDIT_SCORE	SEASO
14	1000.0	0	0	0	620	
17	0.0	0	1	0	519	
27	0.0	0	6	0	539	
29	0.0	0	1	0	535	
47	0.0	0	5	0	563	
...	...	...	...	...	...	
29753	0.0	0	1	1	650	
29766	0.0	0	5	0	550	
29769	5000.0	1	0	0	699	
29789	18500.0	0	0	0	813	
29795	13000.0	0	0	0	580	

3305 rows × 21 columns

In [ ]:

```
# Split the data into train and test sets
train_set_copy = df2.loc[(df2['MONTH_END_DATE'] >= '2013-01-01') & (df2['MONTH_END_DAT

test_set_1_copy = df2.loc[(df2['MONTH_END_DATE'] >= '2018-01-01') & (df2['MONTH_END_DA
test_set_2_copy = df2.loc[(df2['MONTH_END_DATE'] >= '2020-04-01') & (df2['MONTH_END_DA
test_set_3_copy = df2.loc[(df2['MONTH_END_DATE'] >= '2021-01-01') & (df2['MONTH_END_DA
test_set_4_copy = df2.loc[(df2['MONTH_END_DATE'] >= '2022-01-01') & (df2['MONTH_END_DA

# Print
print("Train Set Copy Shape:", train_set_copy.shape)
print("Test Set 1 copy Shape:", test_set_1_copy.shape)
print("Test Set 2 copy Shape:", test_set_2_copy.shape)
print("Test Set 3 copy Shape:", test_set_3_copy.shape)
print("Test Set 4 copy Shape:", test_set_4_copy.shape)
```

```
Train Set Copy Shape: (1365, 21)
Test Set 1 copy Shape: (1420, 21)
Test Set 2 copy Shape: (162, 21)
Test Set 3 copy Shape: (155, 21)
Test Set 4 copy Shape: (38, 21)
```

Dropping MED

In [ ]:

```
# Drop 'MONTH_END_DATE' column from train_set
train_set_copy = train_set_copy.drop('MONTH_END_DATE', axis=1)

# Drop 'MONTH_END_DATE' column from test_set_1
test_set_1_copy = test_set_1_copy.drop('MONTH_END_DATE', axis=1)
test_set_1_copy_X = test_set_1_copy.drop(['Recovery','predicted_Recovery', 'TrueLGD',
test_set_1_copy_X = sm.add_constant(test_set_1_copy_X)
# Drop 'MONTH_END_DATE' column from test_set_2
test_set_2_copy = test_set_2_copy.drop('MONTH_END_DATE', axis=1)
test_set_2_copy_X = test_set_2_copy.drop(['Recovery','predicted_Recovery', 'TrueLGD',
test_set_2_copy_X = sm.add_constant(test_set_2_copy_X)
# Drop 'MONTH_END_DATE' column from test_set_3
```

```
test_set_3_copy = test_set_3_copy.drop('MONTH_END_DATE', axis=1)
test_set_3_copy_X = test_set_3_copy.drop(['Recovery','predicted_Recovery', 'TrueLGD',
test_set_3_copy_X = sm.add_constant(test_set_3_copy_X)
# Drop 'MONTH_END_DATE' column from test_set_4
test_set_4_copy = test_set_4_copy.drop('MONTH_END_DATE', axis=1)
test_set_4_copy_X = test_set_4_copy.drop(['Recovery','predicted_Recovery', 'TrueLGD',
test_set_4_copy_X = sm.add_constant(test_set_4_copy_X)
```

In [ ]:
```
test_set_1_copy_X
```

Out[ ]:

	const	LIMIT	PRODUCT_NAME	TIMES_DELQ_30_DAYS	TIMES_DELQ_90_DAYS	CREDIT_SCORE
87	1.0	0.0	0	20	3	669
88	1.0	0.0	0	23	2	607
199	1.0	0.0	1	5	0	635
250	1.0	7500.0	0	1	0	667
292	1.0	0.0	0	1	0	720
...	...	...	...	...	...	...
29708	1.0	0.0	0	15	0	595
29720	1.0	5000.0	0	0	0	647
29742	1.0	0.0	0	5	3	508
29743	1.0	0.0	0	1	0	559
29789	1.0	18500.0	0	0	0	813

1420 rows × 17 columns

In [ ]:
```
X = train_set_copy.drop(['Recovery','predicted_Recovery', 'TrueLGD', 'PredictedLGD'],
y = train_set_copy['Recovery']
y = y.clip(lower=0 )
X = sm.add_constant(X)  # Add a constant term (intercept) to the independent variables
print(X.shape)
print(y.shape)

model = sm.OLS(y, X)
results = model.fit()
print(results.summary())
```

```
(1365, 17)
(1365,)
                            OLS Regression Results
==============================================================================
Dep. Variable:               Recovery   R-squared:                       0.400
Model:                            OLS   Adj. R-squared:                  0.392
Method:                 Least Squares   F-statistic:                     56.05
Date:                Sat, 06 Jan 2024   Prob (F-statistic):          1.12e-136
Time:                        17:26:15   Log-Likelihood:                -12322.
No. Observations:                1365   AIC:                         2.468e+04
Df Residuals:                    1348   BIC:                         2.477e+04
Df Model:                          16
Covariance Type:            nonrobust
==================================================================================
===================
                                          coef    std err          t      P>|t|
[0.025      0.975]
----------------------------------------------------------------------------------
-------------------
const                                  -97.9229    501.772     -0.195      0.845   -1
082.261     886.416
LIMIT                                    0.0626      0.019      3.249      0.001
0.025       0.100
PRODUCT_NAME                           188.7502    515.653      0.366      0.714    -
822.818    1200.319
TIMES_DELQ_30_DAYS                      71.7673     24.173      2.969      0.003
24.347     119.187
TIMES_DELQ_90_DAYS                    -202.8603     64.329     -3.153      0.002    -
329.056     -76.665
CREDIT_SCORE                             0.6264      0.847      0.740      0.460
-1.035       2.288
SEASONED_OVER_2YR                      138.0636    140.686      0.981      0.327    -
137.924     414.051
BANKPRUPTCY_FLAG                      -193.2381    327.509     -0.590      0.555    -
835.721     449.244
HAS_DIRECT_DEP                          27.1012    138.544      0.196      0.845    -
244.684     298.886
CALC_CURRENT_MONTH_END_SHARE_BALANCE     0.0253      0.073      0.349      0.727
-0.117       0.168
NEG_SHARE_BALANCE                     -174.2373    245.505     -0.710      0.478    -
655.850     307.375
LOYALTY_DUMMY_MEMBER                   -17.1410    128.666     -0.133      0.894    -
269.549     235.267
CALC_PRIOR_MONTH_END_SHARE_BALANCE      -0.0370      0.068     -0.547      0.584
-0.170       0.096
INT_ACCRUED                              0.5704      0.732      0.779      0.436
-0.866       2.007
new_england                            398.0522    129.094      3.083      0.002
144.805     651.299
Diff_OPEN_DATE_CREDIT_SCORE             -0.0378      0.041     -0.926      0.355
-0.118       0.042
BALANCE                                  0.3204      0.018     17.957      0.000
0.285       0.355
==============================================================================
Omnibus:                      524.916   Durbin-Watson:                   1.974
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            11921.338
Skew:                           1.243   Prob(JB):                         0.00
Kurtosis:                      17.263   Cond. No.                     6.45e+04
==============================================================================
```

```
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly spec
ified.
[2] The condition number is large, 6.45e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

In [ ]:
```
train_set_predict = results.predict(X)

column1 = 1 - (train_set_copy['Recovery'] / train_set_copy['BALANCE'])
column2 = 1 - (train_set_predict/ train_set_copy['BALANCE'])

# Clip values to be between 0 and 1
column1 = column1.clip(0, 1)
column2 = column2.clip(0, 1)

# Mean Squared Error (MSE)
mse = np.mean((column1 - column2)**2)

# Print the MSE result
print("Mean Squared Error (MSE):", mse)

# Calculate Sum of Squared Residuals (SSR)
ssr = ((column1 - column2)**2).sum()

# Print the SSR result
print("Sum of Squared Residuals (SSR):", ssr)

# Calculate Total Sum of Squares (SST)
sst = ((column1 - column1.mean())**2).sum()

# Print the SST result
print("Total Sum of Squares (SST):", sst)

# Calculate R-squared (R²)
r_squared = 1 - (ssr / sst)

# Print the R-squared result
print("R-squared (R²):", r_squared)
```

```
Mean Squared Error (MSE): 0.13778740750127091
Sum of Squared Residuals (SSR): 188.07981123923497
Total Sum of Squares (SST): 196.58255908042452
R-squared (R²): 0.043252808799334796
```

In [ ]:
```
test_set_1_copy_pred = results.predict(test_set_1_copy_X)

column1 = 1 - (test_set_1_copy['Recovery'] / test_set_1_copy['BALANCE'])
column2 = 1 - (test_set_1_copy_pred/ test_set_1_copy['BALANCE'])

# Clip values to be between 0 and 1
column1 = column1.clip(0, 1)
column2 = column2.clip(0, 1)

## Mean Squared Error (MSE)
mse = np.mean((column1 - column2)**2)

# Print the MSE result
print("Mean Squared Error (MSE):", mse)

## Calculate Sum of Squared Residuals (SSR)
```

```
ssr = ((column1 - column2)**2).sum()

# Print the SSR result
print("Sum of Squared Residuals (SSR):", ssr)

## Calculate Total Sum of Squares (SST)
sst = ((column1 - column1.mean())**2).sum()

# Print the SST result
print("Total Sum of Squares (SST):", sst)

## Calculate R-squared (R²)
r_squared = 1 - (ssr / sst)

# Print the R-squared result
print("R-squared (R²):", r_squared)
```

```
Mean Squared Error (MSE): 0.15824701092518045
Sum of Squared Residuals (SSR): 224.71075551375606
Total Sum of Squares (SST): 197.23133443861644
R-squared (R²): -0.13932583863185255
```

In [ ]:

```
test_set_2_copy_pred = results.predict(test_set_2_copy_X)

column1 = 1 - (test_set_2_copy['Recovery'] / test_set_2_copy['BALANCE'])
column2 = 1 - (test_set_2_copy_pred/ test_set_2_copy['BALANCE'])

# Clip values to be between 0 and 1
column1 = column1.clip(0, 1)
column2 = column2.clip(0, 1)

## Mean Squared Error (MSE)
mse = np.mean((column1 - column2)**2)

# Print the MSE result
print("Mean Squared Error (MSE):", mse)

## Calculate Sum of Squared Residuals (SSR)
ssr = ((column1 - column2)**2).sum()

# Print the SSR result
print("Sum of Squared Residuals (SSR):", ssr)

## Calculate Total Sum of Squares (SST)
sst = ((column1 - column1.mean())**2).sum()

# Print the SST result
print("Total Sum of Squares (SST):", sst)

## Calculate R-squared (R²)
r_squared = 1 - (ssr / sst)

# Print the R-squared result
print("R-squared (R²):", r_squared)
```

```
Mean Squared Error (MSE): 0.15221227473062715
Sum of Squared Residuals (SSR): 24.658388506361597
Total Sum of Squares (SST): 24.086365011081455
R-squared (R²): -0.023748851062290743
```

In [ ]:
```
test_set_3_copy_pred = results.predict(test_set_3_copy_X)

column1 = 1 - (test_set_3_copy['Recovery'] / test_set_3_copy['BALANCE'])
column2 = 1 - (test_set_3_copy_pred/ test_set_3_copy['BALANCE'])

# Clip values to be between 0 and 1
column1 = column1.clip(0, 1)
column2 = column2.clip(0, 1)

## Mean Squared Error (MSE)
mse = np.mean((column1 - column2)**2)

# Print the MSE result
print("Mean Squared Error (MSE):", mse)

## Calculate Sum of Squared Residuals (SSR)
ssr = ((column1 - column2)**2).sum()

# Print the SSR result
print("Sum of Squared Residuals (SSR):", ssr)

## Calculate Total Sum of Squares (SST)
sst = ((column1 - column1.mean())**2).sum()

# Print the SST result
print("Total Sum of Squares (SST):", sst)

## Calculate R-squared (R²)
r_squared = 1 - (ssr / sst)

# Print the R-squared result
print("R-squared (R²):", r_squared)
```

```
Mean Squared Error (MSE): 0.16006209775341107
Sum of Squared Residuals (SSR): 24.80962515177872
Total Sum of Squares (SST): 26.009016346758543
R-squared (R²): 0.04611443889262268
```

In [ ]:
```
test_set_4_copy_pred = results.predict(test_set_4_copy_X)

column1 = 1 - (test_set_4_copy['Recovery'] / test_set_4_copy['BALANCE'])
column2 = 1 - (test_set_4_copy_pred/ test_set_4_copy['BALANCE'])

# Clip values to be between 0 and 1
column1 = column1.clip(0, 1)
column2 = column2.clip(0, 1)

## Mean Squared Error (MSE)
mse = np.mean((column1 - column2)**2)

# Print the MSE result
print("Mean Squared Error (MSE):", mse)

## Calculate Sum of Squared Residuals (SSR)
ssr = ((column1 - column2)**2).sum()

# Print the SSR result
print("Sum of Squared Residuals (SSR):", ssr)
```

```
## Calculate Total Sum of Squares (SST)
sst = ((column1 - column1.mean())**2).sum()

# Print the SST result
print("Total Sum of Squares (SST):", sst)

## Calculate R-squared (R²)
r_squared = 1 - (ssr / sst)

# Print the R-squared result
print("R-squared (R²):", r_squared)
```

```
Mean Squared Error (MSE): 0.12192592750027934
Sum of Squared Residuals (SSR): 4.633185245010615
Total Sum of Squares (SST): 5.001580485339814
R-squared (R²): 0.07365576569426546
```

Conclusion

Model 1: Logistic Regression

1. **Factors Impacting Recovery:**
    - Limit (-)
    - Credit Score (-)
    - Bankruptcy Flag (+)
    - Has Direct Deposit (-)
    - Negative Share Balance (-)
    - New England (-)
    - Diff Open Date Credit Score (+)
2. **Interpretation:**
    - (+) Higher variable values increase the chance of recovery.
    - (-) Higher variable values decrease the chance of recovery.

Model 2: Linear Regression

1. **Factors Impacting Recovery Amount:**
    - Limit (+)
    - Times Delinquent 30 Days (+)
    - Times Delinquent 90 Days (-)
    - New England (+)
    - Balance (+)
2. **Interpretation:**
    - (+) Higher variable values result in higher recovery $ amounts.
    - (-) Higher variable values result in lower recovery $ amounts.

Key Observations:

1. The likelihood of recovery decreases with an increase in the credit limit; however, in cases of successful recovery, a higher credit limit correlates with a greater amount reclaimed.
2. Members in New England who default face a diminished chance of recovery; yet, when recovery occurs, the amount reclaimed tends to be higher.
3. The subpar performance of the models underscores the challenging nature of this problem, indicating the necessity for an improved set of predictors.
4. Models struggle to perform effectively during the COVID years due to the unique challenges and characteristics posed by the pandemic.
5. The Negative R^2 in the Linear Regression model highlights its fit to the training data but faltering severely when applied to the testing data.